UPDATE 5108169

Sumário

Guia de Estudo: Suporte a Extração de Documentos DOCX e PDF ... 3
Secção 1: Questionário de Avaliação (Quiz) ... 3
Secção 2: Chave de Respostas (Answer Key) ... 3
Secção 3: Questões de Desenvolvimento (Essay Questions) ... 4
Secção 4: Glossário de Termos-Chave ... 5
Briefing Técnico: Suporte para Processamento de Documentos DOCX e PDF (`commit 5108169`) ... 6
Resumo Executivo ... 6
Análise Detalhada dos Temas e Implementações ... 6
1. Atualizações de Dependências (`package.json` e `bun.lock`) ... 6
2. Motor de Limpeza de Texto (`cleanExtractedText`) ... 6
3. Mecanismos de Extração de Documentos ... 7
4. Reformulação da Interface do Utilizador (`AntiplagiarismTccView.tsx`) ... 7
5. Declarações de Tipo TypeScript (`mammoth.d.ts`) ... 8
Mapeamento de Ficheiros Alterados no Commit ... 8
Manual de Conceitos: Extração e Limpeza de Texto no Navegador (PDF & DOCX) ... 9
1. Introdução: O Desafio do Processamento de Documentos no Lado do Cliente ... 9
2. Comparativo de Bibliotecas: PDF.js vs. Mammoth.js ... 9
3. Anatomia do Fluxo de Extração Técnica ... 10
4. O Pipeline de Limpeza e Normalização de Texto (`cleanExtractedText`) ... 11
5. Indicadores de Qualidade e Interface do Utilizador ... 12
6. Conclusão e Resumo de Aprendizagem ... 13
Guia de Aprendizagem: O Ciclo de Vida do Carregamento e Extração de Documentos em React ... 13
1. Introdução e Visão Geral da Arquitetura ... 14
2. Fase 1: Captura, Validação e Estado Inicial na Interface (`AntiplagiarismTccView.tsx`) ... 14
3. Fase 2: Roteamento Inteligente de Formatos na Camada de Serviço (`antiplagiarismService.ts`) ... 16
4. Fase 3: Extração Especializada e Sanitização de Texto (`cleanExtractedText`) ... 18
5. Fase 4: Atualização de Estado, Métricas e Pré-visualização na UI ... 20
Lista de Verificação de Aprendizagem (Checklist do Estudante) ... 22
Relatório de Engenharia: Migração da Extração de Documentos de Regex Heurístico para Processamento Robusto com PDF.js e Mammoth.js ... 22
1. Contexto Técnico e Objetivos de Engenharia ... 22
2. Análise Comparativa de Arquitetura: Módulo Legado vs. Nova Solução ... 23
3. Implementação Técnica e Mecanismos da Nova Solução ... 24

4. Robustez, Gestão de Erros e Impacto na Qualidade para Auditoria 27

5. Conclusão e Recomendações de Engenharia 29

Documento de Arquitetura de Software: Subsistema de Ingestão e Processamento de Documentos Anti-Plágio 29

1. Visão Geral e Objetivos do Subsistema de Ingestão 29

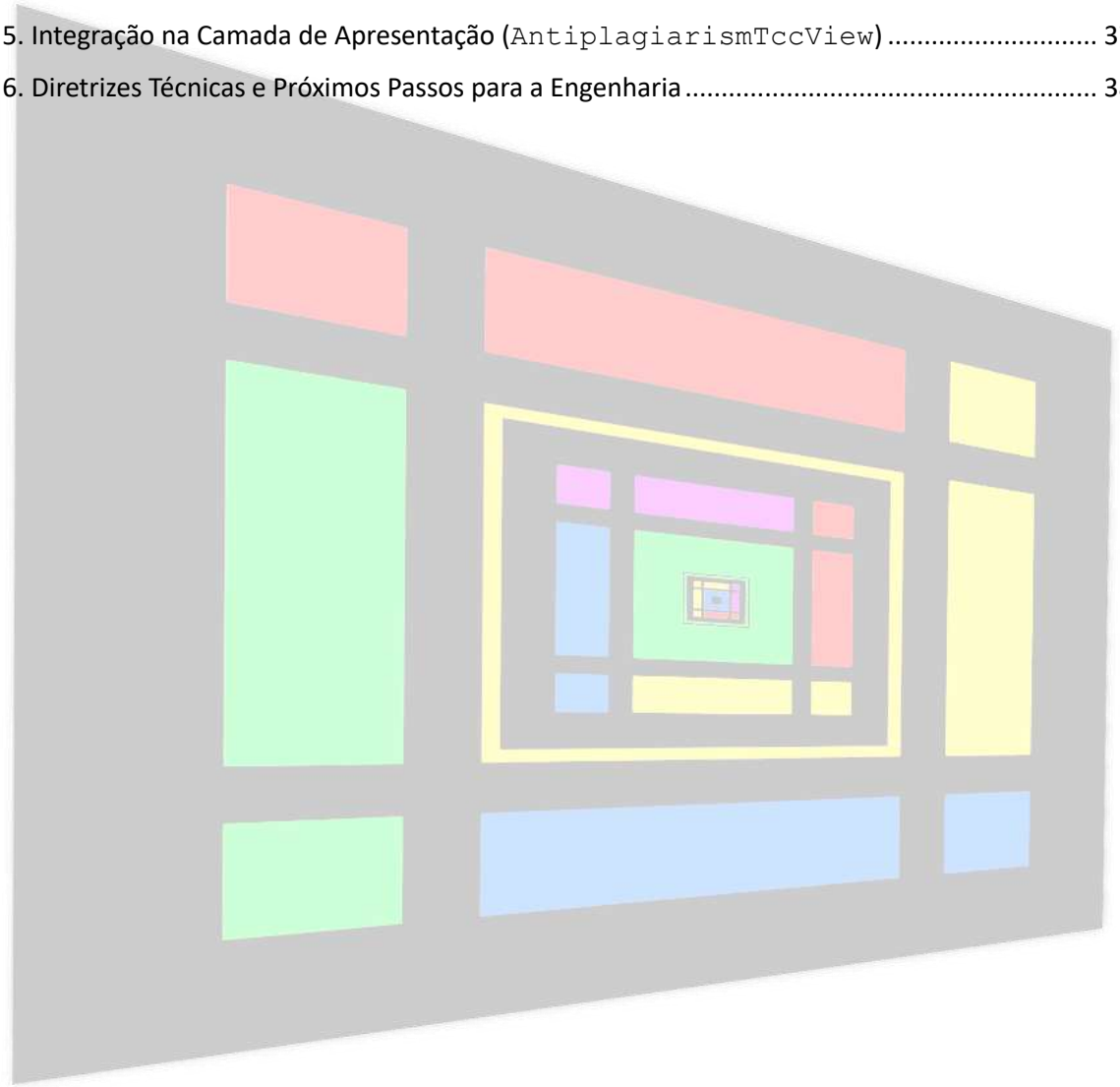
2. Infraestrutura, Dependências e Gestão de Tipos 31

3. Arquitetura do Serviço Principal (`AntiplagiarismService`) 33

4. Estratégia de Estimativa de Páginas ABNT e Métricas 37

5. Integração na Camada de Apresentação (`AntiplagiarismTccView`) 37

6. Diretrizes Técnicas e Próximos Passos para a Engenharia 38



Guia de Estudo: Suporte a Extração de Documentos DOCX e PDF

Este guia de estudo oferece uma revisão aprofundada das alterações introduzidas no repositório `Wiki-alternative` (commit `5108169`), focando-se na integração das bibliotecas **Mammoth.js** e **PDF.js** para extração de texto de ficheiros Word e PDF no módulo de análise anti-plágio de Trabalhos de Conclusão de Curso (TCC).

Secção 1: Questionário de Avaliação (Quiz)

Responda às seguintes dez questões de resposta curta com base na implementação técnica descrita no código-fonte.

1. Quais foram as duas novas dependências principais adicionadas ao ficheiro `package.json` e quais são as suas respetivas funções no projeto?
2. Como é realizada a estimativa do número de páginas ABNT ao processar ficheiros do Microsoft Word (`.docx`) através do serviço `AntiplagiarismService`?
3. Quais são as etapas de higienização de texto executadas pelo método estático `cleanExtractedText`?
4. Qual é o comportamento do sistema quando um utilizador envia um ficheiro PDF constituído exclusivamente por imagens digitalizadas sem camada de texto selecionável?
5. Como é configurado o *worker* da biblioteca `pdfjs-dist` no serviço de anti-plágio para garantir o seu funcionamento no navegador e qual é a solução de reserva (*fallback*) adotada?
6. Que novos formatos e tipos MIME de ficheiros passaram a ser aceites pelo componente de interface `AntiplagiarismTccView.tsx`?
7. Como é que o método `extractTextFromPdfFile` utiliza a coordenada vertical (Y) dos elementos de texto para identificar quebras de linha e a formação de parágrafos?
8. Qual foi a finalidade da criação do ficheiro `src/types/mammoth.d.ts` no projeto?
9. De que forma o método `extractTextFromDocument` trata ficheiros cuja extensão ou tipo MIME não permitam determinar de forma imediata se o formato é PDF ou Word?
10. Que alterações foram efetuadas nos estados do React no componente `AntiplagiarismTccView.tsx` para substituir a antiga lógica exclusiva para PDFs?

Secção 2: Chave de Respostas (Answer Key)

1. **Dependências Adicionadas:** As bibliotecas adicionadas foram a `mammoth` (versão `^1.13.0`) e a `pdfjs-dist` (versão `^6.4.299`). A biblioteca `mammoth` é utilizada para converter e extrair texto limpo de ficheiros Word (`.docx`), enquanto a `pdfjs-dist` (PDF.js da Mozilla) é responsável pelo *parsing* e extração de texto página a página em ficheiros PDF.
2. **Estimativa de Páginas ABNT em DOCX:** A estimativa de páginas para ficheiros `.docx` é calculada dividindo o número total de palavras do texto limpo por 280, estabelecendo o limite mínimo de uma página. Este cálculo baseia-se na média estipulada pela norma ABNT NBR 14724 para páginas padrão de trabalhos académicos.

3. **Etapas de Limpeza do Texto (`cleanExtractedText`):** O método remove caracteres nulos e de controlo não-imprimíveis, normaliza quebras de linha Windows/Mac e elimina resquícios de *streams* ou artefatos binários de PDF. Além disso, reúne palavras divididas por hífen no final de linha (ex.: "sustenta-\nbilidade" para "sustentabilidade") e consolida linhas simples dentro do mesmo parágrafo, separando parágrafos com quebras duplas (`\n\n`).
4. **Tratamento de PDFs Digitalizados (Sem Camada de Texto):** Se o PDF for processado pelo PDF.js mas não contiver texto selecionável, o sistema deteta a ausência de conteúdo e devolve uma mensagem informativa. A mensagem notifica o utilizador de que o ficheiro é baseado em imagens digitalizadas e orienta-o a colar o texto do TCC diretamente no campo de edição manual.
5. **Configuração do Worker do PDF.js:** O sistema tenta atribuir ao `GlobalWorkerOptions.workerSrc` o caminho do *worker* local (`pdfjs-dist/build/pdf.worker.min.mjs?url`). Caso ocorra uma exceção no ambiente de execução, é acionado um *fallback* que carrega o *worker* diretamente da CDN externa `unpkg.com` correspondente à versão exata do `pdfjsLib`.
6. **Formatos e Tipos MIME Aceites:** O componente passou a aceitar extensões `.pdf`, `.docx`, `.doc` e `.txt`. Os tipos MIME suportados incluem `application/pdf`, tipos associados ao Word (`application/vnd.openxmlformats-officedocument.wordprocessingml.document` e `application/msword`) e texto simples (`text/plain`).
7. **Estruturação por Coordenada Vertical (Y):** Ao percorrer os elementos de texto extraídos de uma página, o algoritmo compara a coordenada vertical atual (`currentY`) com a anterior (`lastY`). Se a variação vertical absoluta for superior a 5 unidades, é inserida uma quebra de linha; se for superior a 14 unidades, o algoritmo reconhece um novo parágrafo e insere uma quebra dupla (`\n\n`).
8. **Objetivo do Ficheiro `mammoth.d.ts`:** O ficheiro fornece as definições de tipos TypeScript para a biblioteca `mammoth`, que não possui tipos nativos. Ele declara as interfaces `MammothOptions`, `MammothMessage` e `MammothResult`, além de tipar funções como `extractRawText`, `convertToHtml` e `convertToMarkdown`.
9. **Mecanismo de Resiliência no Roteamento de Ficheiros:** Quando as extensões e tipos MIME não são conclusivos, o método lê os primeiros 5 *bytes* do ficheiro para verificar se começam com o cabeçalho `%PDF`. Se a assinatura for confirmada, o ficheiro é enviado para o PDF.js; caso contrário, tenta-se o *parsing* via `mammoth.js` antes de lançar uma exceção de formato não suportado.
10. **Mudanças nos Estados do React:** O estado `isExtractingPdf` foi substituído por `isExtractingDocument` para refletir um carregamento genérico de documentos. Adicionalmente, foi criado o estado `fileFormat` (que assume os valores `'pdf'`, `'docx'`, `'txt'` ou `null`) para gerir os distintivos visuais e métricas exibidas na interface.

Secção 3: Questões de Desenvolvimento (Essay Questions)

Utilize estas questões para aprofundar a compreensão dos conceitos arquiteturais do projeto. Não são fornecidas respostas para estas perguntas.

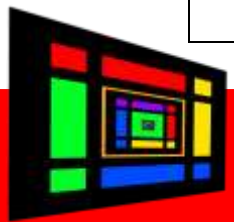
1. Analise a transição da antiga abordagem de extração heurística de PDF baseada em expressões regulares (`BT...ET` / `Tj` / `TJ`) para a utilização de bibliotecas consolidadas

(`pdfjs-dist` e `mammoth`). Quais são os ganhos em termos de robustez e manutenção do código?

2. Explique a importância da função `cleanExtractedText` na preparação de dados textuais para auditoria ABNT e envio para APIs de Inteligência Artificial (como o Gemini). Como é que o "ruído" de formatação num documento pode impactar a verificação anti-plágio?
3. Compare as estratégias de estimativa do número de páginas entre ficheiros PDF (baseada em páginas reais extraídas) e ficheiros Word `.docx` (baseada na contagem de palavras e na norma ABNT NBR 14724). Quais são as vantagens e limitações de cada abordagem?
4. Avalie o padrão de tolerância a falhas implementado no método `extractTextFromDocument` em relação à deteção de formatos de ficheiro (extensão, MIME type e *magic bytes*). De que forma esta arquitetura previne erros no lado do cliente?
5. Discuta as melhorias de experiência de utilizador (UX) introduzidas no componente `AntiplagiarismTccView.tsx`, focando-se no feedback visual em tempo real, distintivos de formato de ficheiro e pré-visualização do texto limpo.

Secção 4: Glossário de Termos-Chave

Termo	Definição do Contexto
ABNT NBR 14724	Norma técnica brasileira que especifica a apresentação de trabalhos académicos; utilizada no projeto como parâmetro de conversão de ~280 palavras por página padrão.
AntiplagiarismService	Classe de serviço estática em TypeScript (`src/services/antiplagiarismService.ts`) que centraliza a lógica de leitura, extração e purificação de ficheiros submetidos para auditoria.
AntiplagiarismTccView	Componente visual React (`src/components/AntiplagiarismTccView.tsx`) responsável por renderizar a interface de carregamento, pré-visualização de texto e métricas do TCC.
bun.lock	Ficheiro de bloqueio de dependências gerado pelo gestor de pacotes Bun, atualizado no commit para registar as versões exatas do `mammoth` e `pdfjs-dist`.
cleanExtractedText	Função utilitária de higienização textual que elimina caracteres de controlo, artefatos binários de PDF, junta hífens de fim de linha e padroniza parágrafos.
GlobalWorkerOptions	Objeto de configuração da biblioteca `pdfjs-dist` utilizado para definir o caminho do ficheiro de script do *worker* executado em segundo plano.
Mammoth.js (`mammoth`)	Biblioteca JavaScript projetada para extrair texto limpo de documentos no formato Word (`.docx`), convertendo a estrutura interna OpenXML sem preservar estilos desnecessários.
PDF.js (`pdfjs-dist`)	Biblioteca desenvolvida pela Mozilla para parsing, parsing de objetos e extração do conteúdo de documentos no formato PDF diretamente no navegador.



pdf.worker.min.mjs	Script de *Web Worker* executado numa *thread* secundária para realizar o processamento pesado de ficheiros PDF sem bloquear a interface de utilizador (UI).
Type Definition (`.d.ts`)	Ficheiro de declarações em TypeScript (`mammoth.d.ts`) que atribui tipos e interfaces a bibliotecas JavaScript puras que não disponibilizam tipos nativos.

Briefing Técnico: Suporte para Processamento de Documentos DOCX e PDF (`commit 5108169`)

Resumo Executivo

O *commit* `51081691655d82ffaa0e2cf7fec4082a0f7a41dd` (no repositório `WazzimaGiygg/Wiki-alternative`) introduz a capacidade de extrair e analisar texto de ficheiros Microsoft Word (`.docx`) e PDF (`.pdf`) na ferramenta de análise antiplágio voltada para Trabalhos de Conclusão de Curso (TCC).

A atualização substitui métodos heurísticos limitados de extração de PDF por bibliotecas robustas e especializadas — **PDF.js** (da Mozilla) e **Mammoth.js**. Além disso, implementa um motor centralizado de limpeza e normalização de texto (`cleanExtractedText`), concebido para eliminar artefatos de formatação, códigos binários e quebras de linha artificiais, garantindo a coesão dos parágrafos antes do envio para a API do Gemini e verificação de normas ABNT.

Análise Detalhada dos Temas e Implementações

1. Atualizações de Dependências (`package.json` e `bun.lock`)

Para suportar os novos formatos de ficheiro, foram adicionadas duas dependências principais ao projeto, resultando em 326 adições e 102 remoções no código do repositório:

- **`mammoth`** (`^1.13.0`): Utilizada para a conversão e extração de texto puro a partir de documentos Word (`.docx`).
- **`pdfjs-dist`** (`^6.4.299`): Biblioteca oficial da Mozilla para processamento estruturado de documentos PDF.

2. Motor de Limpeza de Texto (`cleanExtractedText`)

A classe `AntiplagiarismService` passou a contar com o método estático `cleanExtractedText`, responsável por padronizar o conteúdo extraído. O processo de higienização segue os seguintes passos:

- **Remoção de Caracteres Especiais**: Elimina caracteres nulos e de controlo não imprimíveis através da expressão regular `[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]`, preservando tabulações (`\t`) e quebras de linha (`\n`).
- **Normalização de Quebras de Linha**: Padroniza as quebras dos sistemas Windows e Mac (`\r\n` e `\r`) para `\n`.
- **Remoção de Artefatos de PDF**: Limpa resquícios de *streams* ou blocos binários (como `%PDF-`, `obj` e `stream`).
- **Deshifenização Automática**: Une palavras divididas no final de linhas (ex.: `"sustenta-\nbilidade"` converte-se em `"sustentabilidade"`).
- **Reconstrução de Parágrafos**: Une linhas contínuas pertencentes ao mesmo parágrafo e consolida a separação entre parágrafos com quebras duplas (`\n\n`).

3. Mecanismos de Extração de Documentos

Processamento de PDF (`extractTextFromPdfFile`)

- Configura o *worker* do PDF.js de forma segura via `pdfjs-dist/build/pdf.worker.min.mjs?url` (com *fallback* dinâmico para a CDN `unpkg`).
- Percorre o documento página a página, mapeando as posições verticais (Y) dos elementos de texto:
  - Diferenças na coordenada Y superiores a 5 unidades geram uma nova linha (`\n`).
  - Diferenças superiores a 14 unidades identificam a transição de parágrafo (`\n\n`).
- Inclui tratamento para PDFs digitalizados (constituídos apenas por imagens escaneadas sem camada de texto selecionável), emitindo um aviso explicativo para o utilizador.

Processamento de Word (`extractTextFromDocxFile`)

- Utiliza a função `mammoth.extractRawText` a partir do `ArrayBuffer` do ficheiro.
- Calcula uma estimativa de páginas ABNT baseada na métrica de ~280 palavras por página (em conformidade com a NBR 14724).

Roteamento Unificado (`extractTextFromDocument`)

- Avalia a extensão (`.pdf`, `.docx`, `.doc`, `.txt`) e o tipo MIME do ficheiro enviado.
- Fornece suporte direto a ficheiros de texto simples (`.txt`).
- Executa verificações de segurança por assinatura de ficheiro (procurando pelos bytes `%PDF`) e utiliza *fallbacks* caso a extensão não seja identificada claramente.

4. Reformulação da Interface do Utilizador (`AntiplagiarismTccView.tsx`)

A interface gráfica da vista antiplágio foi ajustada para acomodar a análise multi-formato:

- **Filtros do Input de Ficheiros**: O elemento `<input type="file">` aceita agora os formatos `.pdf`, `.docx`, `.doc` e `.txt`.
- **Estados do Componente**: Substituição do estado `isExtractingPdf` por `isExtractingDocument`, adicionando a variável `fileFormat` (`'pdf'`, `'docx'`, `'txt'`).
- **Identificação Visual (Badges)**:
  - Indicação do motor utilizado (ex.: 📄 `PDF (PDF.js Mozilla)`, 📝 `Word DOCX (Mammoth.js)`).
  - Exibição do tamanho do ficheiro em KB.
  - Relação de páginas reais vs. páginas estimadas ABNT.

  - Contagem total de palavras extraídas.
- **Sanitização de Título**: A extração do nome do documento para o título do TCC remove automaticamente as extensões `.pdf`, `.docx`, `.doc` e `.txt`.

5. Declarações de Tipo TypeScript (`mammoth.d.ts`)

Foi adicionado o ficheiro de declarações `src/types/mammoth.d.ts` para garantir tipagem forte na integração com a biblioteca Mammoth.js:

```
declare module 'mammoth' {
  export interface MammothOptions {
    arrayBuffer?: ArrayBuffer;
    buffer?: any;
    path?: string;
  }

  export interface MammothMessage {
    type: string;
    message: string;
  }

  export interface MammothResult {
    value: string;
    messages: MammothMessage[];
  }

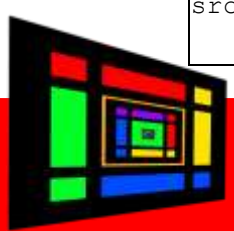
  export function extractRawText(options: MammothOptions):
Promise<MammothResult>;
  export function convertToHtml(options: MammothOptions):
Promise<MammothResult>;
  export function convertToMarkdown(options: MammothOptions):
Promise<MammothResult>;

  const mammoth: {
    extractRawText: typeof extractRawText;
    convertToHtml: typeof convertToHtml;
    convertToMarkdown: typeof convertToMarkdown;
  };

  export default mammoth;
}
```

Mapeamento de Ficheiros Alterados no Commit

Ficheiro	Modificações	Descrição da Alteração
`package.json`	+2 linhas	Adição das bibliotecas `mammoth` (`^1.13.0`) e `pdfjs-dist` (`^6.4.299`).
`bun.lock`	+48 linhas	Atualização do ficheiro de bloqueio de dependências do Bun.
`src/types/mammoth.d.ts`	+29 linhas	Ficheiro de definições de tipos TypeScript para o módulo `mammoth`.



`src/services/antiplagiarismService.ts`	+180 / -75	Substituição da extração legada; adição do `cleanExtractedText` e leitores `PDF.js` / `Mammoth.js`.
`src/components/AntiplagiarismTccView.tsx`	+67 / -27	Atualização da UI para upload, badges informativas e pré-visualização de texto limpo.

Manual de Conceitos: Extração e Limpeza de Texto no Navegador (PDF & DOCX)

Objetivo Central: Capacitar programadores a compreender os fundamentos técnicos da extração, parsing e higienização de dados de texto a partir de documentos PDF e Word (`.docx`) executados inteiramente no navegador (*client-side*), preparando a informação para verificação de plágio e auditoria académica.

1. Introdução: O Desafio do Processamento de Documentos no Lado do Cliente

A extração de texto *client-side* consiste no processamento de ficheiros diretamente no navegador do utilizador, sem a necessidade de enviar o ficheiro bruto para um servidor central antes da leitura inicial. Numa aplicação de análise documental — como uma ferramenta de verificação antiplágio para Trabalhos de Conclusão de Curso (TCCs), monografias e artigos académicos —, essa abordagem transforma radicalmente a arquitetura da aplicação.

Adotar o processamento no cliente oferece três benefícios fundamentais:

- **Privacidade e Segurança dos Dados:** Os documentos académicos do utilizador são lidos e processados diretamente na memória local do navegador, garantindo que o conteúdo original não precisa de ser armazenado em servidores de terceiros antes da higienização e análise.
- **Velocidade e Resposta Imediata:** Elimina-se a latência de rede associada ao *upload* de ficheiros pesados para um servidor, permitindo que a extração de texto e o *feedback* visual na interface sejam quase instantâneos.
- **Redução de Custos de Infraestrutura:** Ao transferir a carga computacional de *parsing* e descompressão de ficheiros para o dispositivo do utilizador, reduz-se drasticamente o consumo de CPU e a largura de banda nos servidores de *backend*.

Para garantir o suporte a múltiplos formatos de ficheiro no navegador, é fundamental recorrer a bibliotecas especializadas capazes de interpretar as estruturas internas de documentos PDF e Word.

2. Comparativo de Bibliotecas: PDF.js vs. Mammoth.js

A leitura de documentos no navegador exige o manuseamento de dois formatos com arquiteturas completamente distintas: o PDF (orientado à representação visual fixa) e o Word

`.docx` (orientado à marcação estrutural de texto). Para solucionar este desafio, a solução integra as bibliotecas `pdfjs-dist` (PDF.js da Mozilla) e `mammoth` (Mammoth.js).

Na integração de processamento para modelos de linguagem (como a API do Gemini) e auditoria ABNT, a escolha do método de leitura é crucial. O Mammoth.js exporta métodos como `convertToHtml`, `convertToMarkdown` e `extractRawText`. A solução adota estritamente o `mammoth.extractRawText`, pois a conversão para HTML ou Markdown introduz marcadores e *tags* de formatação desnecessários. A extração de texto puro economiza orçamento de *tokens* no envio para LLMs e previne contagens falsas de palavras causadas por elementos de marcação.

Biblioteca / Ferramenta	Formato Suportado	Mecanismo Interno de Leitura	Cálculo/Estimativa de Páginas
PDF.js (`pdfjs-dist`)	Ficheiros PDF (`.pdf`)	Leitura de objetos vectoriais e coordenadas visuais de texto via *Web Worker* em blocos de páginas.	**Páginas Reais:** Retorna o número exato de páginas existentes no documento PDF (`pdfDoc.numPages`).
Mammoth.js (`mammoth`)	Ficheiros Word (`.docx`)	Conversão e extração de texto bruto (`extractRawText`) a partir de dados em `ArrayBuffer`.	**Estimativa ABNT:** Fórmula \max(1, \lceil \text{wordCount} / 280 \rceil) baseada na norma ABNT NBR 14724.

A diferença fundamental entre os formatos reside na forma como a informação é armazenada: enquanto o formato `.docx` preserva parágrafos e fluxos de texto contínuos, o PDF distribui caracteres por coordenadas num plano gráfico. Compreender esta distinção é indispensável para interpretar o fluxo técnico de extração de dados.

3. Anatomia do Fluxo de Extração Técnica

A extração de dados segue uma sequência estrita desde o momento em que o utilizador seleciona um ficheiro na interface até ao processamento interno no serviço da aplicação.

Sequência Lógica do Fluxo de Dados:

1. **Interface de Utilizador (`AntiplagiarismTccView.tsx`):** 1.1. O utilizador faz *upload* ou arrasta um ficheiro (`.pdf`, `.docx`, `.doc` ou `.txt`). 1.2. A interface valida o tipo de ficheiro por extensão e MIME type (`application/pdf`, `officedocument`, `text/plain`). 1.3. O estado de carregamento (`isExtractingDocument`) é ativado e o ficheiro é enviado para o serviço principal (`AntiplagiarismService.extractTextFromDocument(file)`).
2. **Roteamento e Detenção no Serviço (`AntiplagiarismService.ts`):** 2.1. O método `extractTextFromDocument` inspeciona o nome e o MIME type do ficheiro (`pdf`, `docx`, `txt`). 2.2. **Processamento de Ficheiros de Texto Simples (`.txt`):** Caso o ficheiro possua a extensão `.txt` ou MIME type `text/plain`, o serviço lê o conteúdo diretamente via `file.text()`, aplica a higienização `cleanExtractedText` e calcula a estimativa de páginas ABNT pela fórmula \max(1, \lceil \text{wordCount} / 280 \rceil), retornando o formato `'txt'`. 2.3. **Fallback e Verificação de Assinatura Binária:** Se a extensão não for conclusiva, o serviço inspeciona os primeiros 5 *bytes* do ficheiro (`file.slice(0, 5).text()`). Se contiver a assinatura `%PDF`, encaminha automaticamente para o extrator de PDF; caso contrário, tenta a leitura via Mammoth.js como *fallback*.

3. **Tipagem TypeScript e Configuração de Infraestrutura:** 3.1. **Declaração de Tipos para Mammoth (`src/types/mammoth.d.ts`):** Como a biblioteca `mammoth` não disponibiliza tipos nativos no ecossistema TypeScript, é necessário criar um ficheiro de declaração de módulo (`declare module 'mammoth'`) mapeando as interfaces `MammothOptions`, `MammothResult` e os métodos `extractRawText`, `convertToHtml` e `convertToMarkdown`. 3.2. **Configuração Segura do Worker PDF.js:** Para evitar o bloqueio da *thread* principal da interface durante a leitura de PDFs longos, o `pdfjs-dist` utiliza um *Web Worker*. O serviço é configurado utilizando a sintaxe de URL do Vite (`import pdfjsWorker from 'pdfjs-dist/build/pdf.worker.min.mjs?url'`). Se a resolução local falhar, o código aciona um *fallback* dinâmico apontando para o CDN Unpkg (`https://unpkg.com/pdfjs-dist@${pdfjsLib.version}/build/pdf.worker.min.mjs`).
4. **Extração de Ficheiros PDF (`extractTextFromPdfFile`):** 4.1. O ficheiro é convertido num `ArrayBuffer` e processado via `pdfjsLib.getDocument`. 4.2. O código itera sobre cada página (`getPage`) e obtém os seus itens de texto (`getTextContent`). 4.3. Para reorganizar o texto em linhas e parágrafos, analisa-se a coordenada vertical Y de cada item no plano visual através da matriz de transformação (`item.transform[5]`).

```
Lógica de Posição Vertical (Coordenada Y via transform[5]):
|--- Item de Texto A (Y = 700)
|      Diferença |Y_atual - Y_anterior| <= 5px -> Mesmo segmento / Linha contínua (espaço)
|--- Item de Texto B (Y = 690) -> Diferença = 10px (> 5px)  -> Quebra de Linha (\n)
|--- Item de Texto C (Y = 650) -> Diferença = 40px (> 14px) -> Novo Parágrafo (\n\n)
```

1. **Extração de Ficheiros Word (`extractTextFromDocxFile`):** 5.1. O ficheiro `.docx` é lido como `ArrayBuffer`. 5.2. A função `mammoth.extractRawText({ arrayBuffer })` extrai o texto bruto. 5.3. O total de palavras é contabilizado e a estimativa de páginas ABNT é calculada pela fórmula \max(1, \lceil \text{wordCount} / 280 \rceil).

Apesar de funcionais, os textos em bruto resultantes de ambos os processos contêm imperfeições estruturais e requerem uma etapa obrigatória de higienização.

4. O Pipeline de Limpeza e Normalização de Texto (`cleanExtractedText`)

O texto extraído diretamente (*raw text*) de ficheiros PDF e Word não pode ser enviado imediatamente para modelos de Inteligência Artificial (como a API do Gemini) ou submetido a auditorias da ABNT. Ficheiros PDF contêm frequentemente artefactos de descompressão, caracteres nulos, quebras de linha no meio de frases e palavras cortadas por hifenização visual de fim de página.

A função estática `cleanExtractedText` em `AntiplagiarismService.ts` executa um *pipeline* rigoroso de 5 passos utilizando expressões regulares (RegEx) para padronizar os dados:

1. **Remoção de caracteres de controlo não-imprimíveis e nulos:**
   - *Código / RegEx:* `.replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]/g, '')`
   - *Descrição:* Elimina bytes nulos e caracteres de controlo do sistema ASCII/ANSI, preservando apenas tabulações (`\t`) e quebras de linha (`\n`).
   - *Antes:* `"Análise\x00 de\x1F Dados"`

- *Depois:* `"Análise de Dados"`

2. **Normalização de quebras de linha:**
   - *Código / RegEx:* `.replace(/\r\n/g, '\n').replace(/\r/g, '\n')`
   - *Descrição:* Padroniza terminadores de linha provenientes do Windows (`\r\n`) ou de edições legadas do Mac (`\r`) para o padrão Unix (`\n`).
   - *Antes:* `"Linha 1\r\nLinha 2\rLinha 3"`
   - *Depois:* `"Linha 1\nLinha 2\nLinha 3"`
3. **Eliminação de resquícios de *streams* e artefactos binários de PDF:**
   - *Código / RegEx:* `.replace(/^(?:%PDF-[\d.]+|<<\/[\w\s/]+>>|obj\b[\s\S]*?endobj|stream[\s\S]*?endstream)/gm, '')`
   - *Descrição:* Remove cabeçalhos de ficheiro, dicionários e blocos de declaração binária de PDF que tenham escapado à conversão.
   - *Antes:* `"%PDF-1.5 << /Type /Page >> stream Texto do TCC endstream"`
   - *Depois:* `"Texto do TCC"`
4. **Reconstrução de palavras hifenizadas no final de linha:**
   - *Código / RegEx:* `.replace(/([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])/g, '$1$2')`
   - *Descrição:* Junta palavras que foram divididas por hifenização gráfica de quebra de linha visual no PDF.
   - *Antes:* `"sustenta-\nção do argumento e desen-\nvolvimento"`
   - *Depois:* `"sustentação do argumento e desenvolvimento"`
5. **Consolidação de parágrafos contínuos e eliminação de espaços excessivos:**
   - *Código / RegEx:*

```
.split(/\n\s*\n+/)
.map((paragraph) => {
  return paragraph
    .split('\n')
    .map((l) => l.trim())
    .filter(Boolean)
    .join(' ')
    .replace(/[ \t\u00A0]+/g, ' ')
    .trim();
})
.filter((p) => p.length > 0)
.join('\n\n');
```

   - *Descrição:* Divide o texto por quebras duplas (parágrafos), unifica linhas simples pertencentes ao mesmo parágrafo separando palavras por um único espaço, elimina espaços em branco e recria parágrafos coesos separados estritamente por `\n\n`.
   - *Antes:* `"Este é um parágrafo\nque foi quebrado no PDF.\n\n\n\nE este é outro."`
   - *Depois:* `"Este é um parágrafo que foi quebrado no PDF.\n\nE este é outro."`

Este tratamento de dados garante que a contagem de palavras e a análise de coerência posterior operem sobre um texto limpo e uniforme, refletindo diretamente na qualidade da experiência apresentada ao utilizador.

5. Indicadores de Qualidade e Interface do Utilizador

Após a extração e limpeza, a aplicação fornece *feedback* imediato ao utilizador através de componentes visuais e metadados informativos na interface (`AntiplagiarismTccView.tsx`):

- **Identificação do Extrator (*Badges* de Formato):** A interface inspeciona o estado `fileFormat` para indicar o mecanismo utilizado:
  - `fileFormat === 'pdf'` \rightarrow Exibe o badge `PDF (PDF.js Mozilla)`.
  - `fileFormat === 'docx'` \rightarrow Exibe o badge `Word DOCX (Mammoth.js)`.
  - `fileFormat === 'txt'` \rightarrow Exibe o badge `Documento de Texto`.
- **Diferenciação Condicional de Páginas:** O sistema valida o tipo de ficheiro para renderizar o rótulo de contagem exato:

```
{fileFormat === 'pdf'
  ? `${pdfPagesEstimate} páginas reais`
  : `~${pdfPagesEstimate} páginas ABNT`}
```

Isto garante ao utilizador a clareza de que PDFs refletem a contagem exata do ficheiro, enquanto ficheiros Word/TXT exibem a estimativa calculada pela norma NBR 14724 (\approx 280 palavras/página).

- **Métricas de Volume e Prévia do Texto:** Apresenta o tamanho do ficheiro em KB (`(file.size / 1024).toFixed(1)`), o total de palavras extraídas e o número aproximado de parágrafos higienizados prontos para verificação, acompanhados pelo indicador visual `CheckCircle2`.
- **Tratamento Arquitetural de PDFs Digitalizados (Sem Camada de Texto):** Do ponto de vista de arquitetura web, o PDF.js extrai texto lendo as tabelas de fontes vectoriais e mapas de glifos do documento. Quando um PDF é fruto de uma digitalização ou *scanner* (composto apenas por imagens rasterizadas PNG/JPEG), não existem tabelas de caracteres legíveis. Nestes casos, o sistema identifica que o texto resultante está vazio e exibe uma mensagem orientadora ao utilizador, explicando a limitação e sugerindo a colagem direta do texto na área de edição.

6. Conclusão e Resumo de Aprendizagem

Dominar a extração de dados no lado do cliente é um passo fundamental para o desenvolvimento de aplicações web modernas, rápidas e seguras. Compreender as diferenças entre estruturas rígidas por coordenadas (PDF) e documentos orientados a texto (Word) permite escolher as ferramentas certas e construir algoritmos de limpeza eficientes.

1. Arquitetura de Extração Adequada: Ficheiros PDF exigem reconstituição de fluxo textual por coordenadas verticais (Y) via *Web Workers* (`pdfjs-dist`), enquanto ficheiros Word `.docx` requerem extração direta de texto bruto (`mammoth.extractRawText`) para economizar o orçamento de *tokens* em APIs de IA. **2. Importância do Pipeline de Sanitização:** Texto bruto extraído no cliente contém ruídos binários, hifenizações quebradas e quebras de linha artificiais. O uso de um *pipeline* com expressões regulares (`cleanExtractedText`) é indispensável para garantir dados de entrada de alta qualidade. **3. Transparência na Interface (UX):** A diferenciação clara de metadados — distinguindo páginas reais de PDFs versus estimativas ABNT e alertando sobre PDFs digitalizados sem camada de texto — proporciona uma experiência transparente e profissional ao utilizador.

Guia de Aprendizagem: O Ciclo de Vida do Carregamento e Extração de Documentos em React

UPDATE 5108169

1. Introdução e Visão Geral da Arquitetura

Bem-vindo a este guia prático de aprendizagem! Se estás a dar os primeiros passos na programação web com React e queres compreender como as aplicações modernas processam ficheiros complexos diretamente no navegador, vieste ao lugar certo. O processamento de documentos no lado do cliente (*client-side*) é uma competência fundamental na arquitetura frontend contemporânea, pois permite extrair conteúdo de ficheiros sem sobrecarregar desnecessariamente o servidor nem comprometer a privacidade dos dados.

Neste guia, vais aprender a jornada completa que um ficheiro (seja no formato PDF, Word DOCX ou Texto Puro) percorre: desde o momento em que o utilizador clica ou arrasta o ficheiro para a interface de utilizador (UI), passando pela validação, roteamento de formatos, parsing binário com Web Workers e limpeza de dados, até ao momento final em que o texto limpo e as métricas calculadas são apresentados no ecrã.

A tabela abaixo resume as 4 fases essenciais deste ciclo de vida:

Fase do Ciclo de Vida	Componente / Módulo Responsável	Objetivo Principal para o Utilizador ("So What?")
1. Captura e Validação	`AntiplagiarismTccView.tsx`	Garante que apenas ficheiros válidos são aceites e dá feedback visual imediato ao utilizador.
2. Roteamento Inteligente	`antiplagiarismService.ts`	Identifica o formato real do ficheiro e delega o processamento para os motores apropriados (`PDF.js` ou `Mammoth.js`).
3. Extração e Sanitização	`cleanExtractedText` & Motores de Parsing	Converte streams binários e coordenadas espaciais em texto puro limpo, eliminando artefatos e corrigindo a paginação.
4. Atualização e Exibição	`AntiplagiarismTccView.tsx`	Atualiza a UI com o texto sanitizado, ajusta o título do documento e exibe etiquetas de métricas detalhadas.

Com esta visão geral em mente, vamos examinar o ponto de partida de toda esta jornada: a interação inicial do utilizador na interface React.

2. Fase 1: Captura, Validação e Estado Inicial na Interface (`AntiplagiarismTccView.tsx`)

A primeira fase do ciclo de vida inicia-se na camada de apresentação (`AntiplagiarismTccView.tsx`), onde a interface reage à ação do utilizador — seja através de um clique no elemento `<input type="file">` ou através do gesto de arrastar e largar (*drag-and-drop*).

Durante este evento de carregamento, o React gere ativamente diversos estados locais para garantir uma experiência fluida:

- **`selectedFile`**: Armazena a referência direta ao objeto `File` selecionado pelo utilizador na memória do browser.
- **`fileFormat`**: Regista o formato do ficheiro detetado (aceitando os valores `'pdf'`, `'docx'`, `'txt'` ou `null`).



- **`isExtractingDocument`**: Estado booleano que indica se o processo assíncrono de extração está em execução.
- **`pdfPagesEstimate`**: Guarda a estimativa ou o número real de páginas calculadas para o documento.

Validação em Camadas (Defense-in-Depth)

Uma implementação robusta de carregamento de ficheiros em React deve empregar o princípio de segurança em profundidade (*defense-in-depth*), combinando restrições declarativas na UI com validação programática em tempo de execução (*runtime*):

```
<input
  type="file"
  ref={fileInputRef}
  accept=".pdf,application/pdf,.docx,application/vnd.openxmlformats-
officedocument.wordprocessingml.document,.doc,application/msword,.txt,text/
plain"
  onChange={(e) => e.target.files && handleFileChange(e.target.files[0])}
  className="hidden"
/>
```

1. **Restrição Declarativa na UI (`accept`)**: O atributo `accept` instrui o sistema operativo a filtrar os ficheiros visíveis na janela de seleção do sistema. No entanto, é crucial compreender que a funcionalidade *drag-and-drop* ou a alteração manual de filtros pelo utilizador contornam totalmente este atributo.
2. **Validação Programática (`handleFileChange`)**: Quando o evento de carregamento é disparado, a função `handleFileChange` inspeciona rigorosamente a extensão e o tipo MIME do ficheiro antes de prosseguir:

```
const nameLower = file.name.toLowerCase();
const isPdf = nameLower.endsWith('.pdf') || file.type.includes('pdf');
const isDocx =
  nameLower.endsWith('.docx') ||
  nameLower.endsWith('.doc') ||
  file.type.includes('word') ||
  file.type.includes('officedocument');
const isTxt = nameLower.endsWith('.txt') ||
file.type.includes('text/plain');

if (!isPdf && !isDocx && !isTxt) {
  alert('Por favor, selecione um arquivo no formato PDF (.pdf) ou Word
(.docx).');
  return;
}
```

Se o ficheiro não cumprir nenhum destes critérios, a execução é interrompida imediatamente, prevenindo erros de processamento em etapas posteriores.

Boas Práticas de UI/UX: Sempre que um ficheiro válido é aceite, a aplicação deve alterar imediatamente o estado `isExtractingDocument` para `true`. Isto ativa um indicador visual de carregamento (o ícone `Loader2` da biblioteca `lucide-react` com a classe CSS `animate-spin text-blue-600`), fornecendo feedback visual instantâneo e prevenindo re-submissões acidentais enquanto o documento é processado.



Assim que o ficheiro é validado com sucesso e os estados visuais são ativados, a execução do código é delegada à camada de serviço especializada.

3. Fase 2: Roteamento Inteligente de Formatos na Camada de Serviço (`antiplagiarismService.ts`)

A camada de apresentação invoca o método estático `AntiplagiarismService.extractTextFromDocument(file)`. Este método funciona como um roteador central que inspeciona as propriedades do ficheiro e delega a extração aos métodos privados auxiliares `extractTextFromPdfFile` e `extractTextFromDocxFile`.

A tabela comparativa seguinte detalha a estratégia de extração por tipo de ficheiro:

Formato de Ficheiro	Mecanismo de Deteção (MIME/Extensão)	Biblioteca Externa / API	Estratégia de Fallback/Segurança
PDF (`.pdf`)	`.endsWith('.pdf')` ou `mime.includes('pdf')`	`pdfjs-dist` (PDF.js da Mozilla com Web Worker)	Deteção da assinatura binária `%PDF` nos primeiros 5 bytes.
Word (`.docx` / `.doc`)	`.endsWith('.docx')`, `.endsWith('.doc')`, `word` ou `officedocument`	`mammoth.js` (`mammoth.extractRawText`)	*Fallback* final para `mammoth.js` caso a extensão seja omissa.
Texto Puro (`.txt`)	`.endsWith('.txt')` ou `mime.includes('text/plain')`	API nativa `file.text()`	Leitura direta do fluxo de carateres em texto plano.

Isolamento de Thread Principal com Web Workers (`pdfjs-dist`)

O parsing de ficheiros PDF é uma operação computacionalmente pesada que envolve a descompressão de streams binários e o cálculo de vetores gráficos. Se for executada diretamente na *main thread* do browser, a interface congelará, resultando numa má experiência de utilização (UX). Para evitar este problema, a aplicação configura um Web Worker dedicado:

```
import * as pdfjsLib from 'pdfjs-dist';
import pdfjsWorker from 'pdfjs-dist/build/pdf.worker.min.mjs?url';

// Configura o worker do PDF.js de forma segura com fallback resiliente
if (typeof window !== 'undefined' && pdfjsLib.GlobalWorkerOptions) {
  try {
    pdfjsLib.GlobalWorkerOptions.workerSrc = pdfjsWorker;
  } catch {
    pdfjsLib.GlobalWorkerOptions.workerSrc = `https://unpkg.com/pdfjs-
dist@${pdfjsLib.version}/build/pdf.worker.min.mjs`;
  }
}
```

Ao importar o script do worker utilizando o sufixo `?url` (suportado por empaquetadores modernos como Vite ou Bun), o browser carrega o worker num processo em segundo plano isolado. Se a resolução do pacote local falhar, a aplicação faz um *fallback* gracioso para a CDN da unpkg.

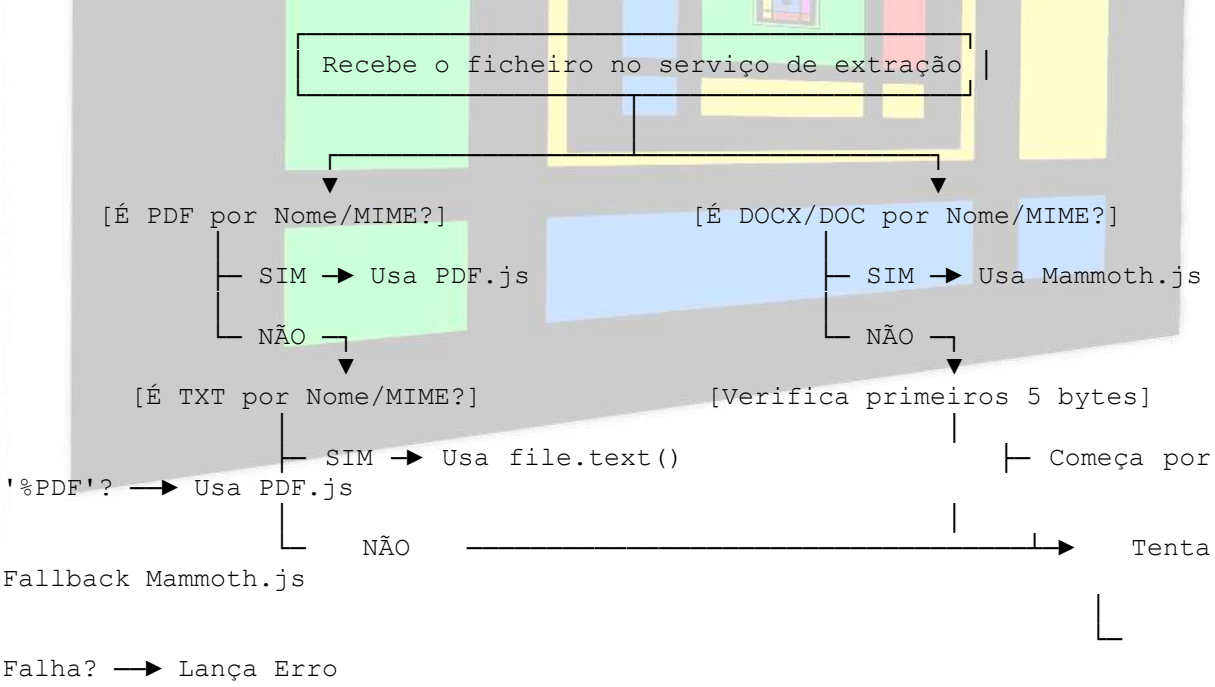
Dica de Arquitetura — Declarações de Tipos Ambientais (`.d.ts`): Nem todas as bibliotecas JavaScript de terceiros incluem definições de tipos TypeScript prontas a usar. Ao integrar bibliotecas como a `mammoth`, podes encontrar erros de compilação do TypeScript. Para resolver isto, cria um ficheiro de declaração de módulo ambiental (ex.: `src/types/mammoth.d.ts`):

```
declare module 'mammoth' {
  export interface MammothOptions {
    arrayBuffer?: ArrayBuffer;
    buffer?: any;
    path?: string;
  }
  export interface MammothResult {
    value: string;
    messages: any[];
  }
  export function extractRawText(options: MammothOptions):
Promise<MammothResult>;
  const mammoth: { extractRawText: typeof extractRawText };
  export default mammoth;
}
```

Esta técnica ensina ao compilador TypeScript a assinatura exata dos métodos exportados sem alterar o código de tempo de execução da biblioteca.

Fluxo de Roteamento e Fallback Binário

Se um ficheiro for enviado sem extensão válida ou com um tipo MIME genérico (`application/octet-stream`), o método `extractTextFromDocument` executa uma verificação de segurança ao nível dos bytes:



1. **Deteção da Assinatura Binária (%PDF)**: O serviço lê o cabeçalho do ficheiro através de `file.slice(0, 5).text()`. Se os primeiros 5 bytes corresponderem a `%PDF`, o ficheiro é processado por `extractTextFromPdfFile`.

2. **Fallback Final**: Se a assinatura binária não for detetada, o roteador tenta a conversão através de `extractTextFromDocxFile` com a biblioteca `mammoth.js`. Se ambas as tentativas falharem, lança uma exceção: *"Formato de arquivo não suportado. Por favor envie um documento no formato PDF (.pdf) ou Word (.docx)."*

No entanto, obter o texto bruto de um ficheiro binário é apenas metade do trabalho; a extração bruta produz ruído que precisa de ser higienizado.

4. Fase 3: Extração Especializada e Sanitização de Texto (`cleanExtractedText`)

A extração de documentos PDF e Word levanta desafios únicos de estruturação de texto. Os ficheiros PDF, em particular, não armazenam parágrafos contínuos; armazenam glifos e coordenadas espaciais bidimensionais (X, Y) numa página.

Reconstrução Espacial de Texto no PDF.js

Dentro do método `extractTextFromPdfFile`, a aplicação itera sobre todas as páginas do documento e analisa a propriedade `transform[5]` (que representa a coordenada vertical Y no espaço 2D do PDF) de cada item retornado por `page.getTextContent()`:

```
let pageStr = '';
let lastY: number | undefined;

for (const item of textContent.items as any[]) {
  if (!item || typeof item.str !== 'string') continue;
  const str = item.str;
  const currentY = item.transform ? item.transform[5] : undefined;

  if (lastY !== undefined && currentY !== undefined && Math.abs(currentY -
lastY) > 5) {
    if (Math.abs(currentY - lastY) > 14) {
      pageStr += '\n\n'; // Mudança significativa de coordenada Y -> Novo
Parágrafo
    } else {
      pageStr += '\n';   // Mudança menor de coordenada Y -> Quebra de Linha
    }
  }   else   if   (pageStr.length   >   0   &&   !pageStr.endsWith('   ')   &&
!pageStr.endsWith('\n')) {
    pageStr += ' ';
  }

  pageStr += str;
  if (currentY !== undefined) lastY = currentY;
}
```

- **$\Delta Y > 5\text{ px}$**: Indica uma mudança de linha simples.
- **$\Delta Y > 14\text{ px}$**: Indica um salto maior entre blocos de texto, traduzido para uma quebra de parágrafo (`\n\n`).

As 5 Etapas da Função `cleanExtractedText`

Após a extração bruta (seja via `pdfjs-dist`, `mammoth.js` ou `file.text()`), o resultado é processado pela função sanitizadora `cleanExtractedText`:

```
static cleanExtractedText(rawText: string): string {
  if (!rawText) return '';

  return rawText
    // 1. Remove caracteres nulos e de controlo não-imprimíveis (mantém \t e
\n)
    .replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]/g, '')
    // 2. Normaliza quebras de linha Windows (\r\n) e Mac antigo (\r)
    .replace(/\r\n/g, '\n')
    .replace(/\r/g, '\n')
    // 3. Remove resquícios de streams binários ou estruturas internas do
PDF
    .replace(/^(?:%PDF-
[\d.]+|<<\/[\w\s/]+>>|obj\b[\s\S]*?endobj|stream[\s\S]*?endstream)/gm, '')
    // 4. Une palavras quebradas por hífen no fim da linha (ex: "sustenta-
\nbilidade" -> "sustentabilidade")
    .replace(/([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])/g, '$1$2')
    // 5. Reconstrói parágrafos contínuos mantendo apenas duplo \n\n entre
blocos
    .split(/\n\s*\n+/)
    .map((paragraph) => {
      return paragraph
        .split('\n')
        .map((l) => l.trim())
        .filter(Boolean)
        .join(' ')
        .replace(/[ \t\u00A0]+/g, ' ')
        .trim();
    })
    .filter((p) => p.length > 0)
    .join('\n\n');
}
```

1. **Eliminação de Caracteres Não-Imprimíveis**: Remove códigos nulos e de controlo invisíveis via expressão regular, preservando apenas tabulações e quebras de linha normais.
2. **Padronização de Quebras de Linha**: Unifica as convenções de terminadores de linha para o padrão Unix (`\n`).
3. **Remoção de Artefatos Binários**: Elimina declarações residuais do cabeçalho ou blocos de stream do PDF (`%PDF-`, `obj...endobj`).
4. **Deshifenização Automática**: Reconecta palavras divididas no final das linhas por questões de justificação de texto (transformando `"sustenta-\nbilidade"` em `"sustentabilidade"`).
5. **Normalização Estrutural de Parágrafos**: Agrupa linhas pertencentes ao mesmo bloco de pensamento, remove espaços duplicados e separa parágrafos distintos com exatamente dois carateres de nova linha (`\n\n`).

Algoritmo de Estimativa de Páginas

A contagem de páginas utiliza duas abordagens consoante a natureza do ficheiro:

- **PDFs**: O número de páginas é fornecido de forma exata pela propriedade `pdfDoc.numPages` da biblioteca `pdfjs-dist`.
- **Word (DOCX) e TXT**: Ficheiros de texto não estruturado não contêm numeração de páginas nativa. Por conseguinte, calcula-se uma estimativa com base na norma ABNT NBR 14724 (~280 palavras por página padrão): \text{páginasEstimadas} = \max\left(1, \left\lceil \frac{\text{totalDePalavras}}{280} \right\rceil\right)

Com o texto sanitizado e as métricas calculadas, os dados regressam à interface do utilizador.

5. Fase 4: Atualização de Estado, Métricas e Pré-visualização na UI

Ao receber o objeto com `{ text, pagesEstimate, format }`, o componente React atualiza os seus estados internos (`setTccText`, `setPdfPagesEstimate`, `setFileFormat`), desativa o indicador de carregamento e atualiza a interface.

Limpeza Inteligente do Título do Documento

Para poupar tempo ao utilizador, o sistema atualiza automaticamente o campo do título do trabalho (`tccTitle`). No entanto, esta atualização só deve ocorrer se o título atual for nulo ou corresponder a um dos valores por omissão da aplicação.

A função verifica **três condições de salvaguarda (*guard conditions*)**:

```
if (
  !tccTitle ||
  tccTitle === 'Trabalho de Conclusão de Curso (TCC)' ||
  tccTitle === 'A Aplicação de Inteligência Artificial na Gestão de Conhecimento'
) {
  const cleanName = file.name
    .replace(/\.(pdf|docx|doc|txt)$/i, '')
    .replace(/[-_]+/g, ' ');
  setTccTitle(cleanName);
}
```

Atenção ao Padrão Regex de Substituição: Observa que a expressão `.replace(/[-_]+/g, ' ')` substitui hífenes e sublinhados por **um espaço em branco** (e não por uma string vazia `''`). Se usássemos uma string vazia, o ficheiro `meu-trabalho-tcc.pdf` seria transformado em `meutrabalhotcc`. A substituição por espaço preserva a separação natural entre palavras, gerando o título `"meu trabalho tcc"`.

Renderização Contextual das Etiquetas Visuais (Badges)

A UI apresenta etiquetas coloridas baseadas no formato extraído e nas métricas do documento:

```
<div className="flex flex-wrap items-center justify-center gap-2 text-xs font-mono">
  {/* Badge de Formato e Extrator */}
  <span className="px-3 py-1 rounded-full bg-blue-50 dark:bg-blue-950 text-blue-700 dark:text-blue-300 border border-blue-200 dark:border-blue-800 font-bold">
    {fileFormat === 'pdf'
      ? '📄 PDF (PDF.js Mozilla)'
      : fileFormat === 'docx'
      ? '📝 Word DOCX (Mammoth.js)'
      : '📄 Documento de Texto'}
  </span>

  {/* Badge de Tamanho */}
```

```
  <span  className="px-3  py-1  rounded-full  bg-slate-100  dark:bg-slate-800
text-slate-700  dark:text-slate-300  border  border-slate-200  dark:border-
slate-700">
    Tamanho: {(selectedFile.size / 1024).toFixed(1)} KB
  </span>

  {/* Badge de Paginação */}
  {pdfPagesEstimate && (
    <span className="px-3 py-1 rounded-full bg-purple-50 dark:bg-purple-950
text-purple-700 dark:text-purple-300 border border-purple-200 dark:border-
purple-800">
      {fileFormat  ===  'pdf'  ?  `${pdfPagesEstimate}  páginas  reais`  :
`~${pdfPagesEstimate} páginas ABNT`}
    </span>
  )}

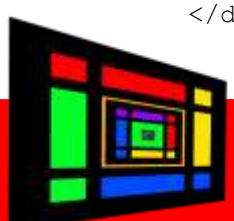
  {/* Badge de Contagem de Palavras */}
  {wordCount > 0 && (
    <span  className="px-3  py-1  rounded-full  bg-emerald-50  dark:bg-emerald-
950  text-emerald-700  dark:text-emerald-300  border  border-emerald-200
dark:border-emerald-800">
      {wordCount} palavras extraídas
    </span>
  )}
</div>
```

Melhores Práticas na Área de Pré-visualização (`<textarea>`)

O texto limpo é carregado num campo de edição configurado para uma leitura confortável e auditável:

```
<div className="space-y-1.5">
  <div className="flex items-center justify-between text-xs text-slate-500">
    <span  className="flex  items-center  gap-1.5  font-medium  text-slate-700
dark:text-slate-300">
      <CheckCircle2 size={13} className="text-emerald-600" />
      <span>
        Texto limpo extraído com{' '}
        <strong>
          {fileFormat  ===  'docx'  ?  'Mammoth.js'  :  fileFormat  ===  'pdf'  ?
'PDF.js (Mozilla)' : 'Extrator'}
        </strong>
        :
      </span>
    </span>
    <span>{wordCount} palavras • ~{paragraphEstimate} parágrafos prontos para
a auditoria</span>
  </div>

  <textarea
    value={tccText}
    onChange={(e) => setTccText(e.target.value)}
    rows={5}
    className="w-full  p-3  text-xs  bg-slate-50  dark:bg-slate-950  border
border-slate-200 dark:border-slate-800 rounded-2xl focus:outline-none font-
sans leading-relaxed text-slate-800 dark:text-slate-200"
    placeholder="O texto limpo extraído do documento aparecerá aqui..."
  />
</div>
```



O uso do ícone `CheckCircle2` em conjunto com a especificação do motor de extração ativo traz transparência e segurança visual ao utilizador. As propriedades CSS `rows={5}` e `leading-relaxed` oferecem uma excelente experiência de leitura e edição no ecrã.

Lista de Verificação de Aprendizagem (Checklist do Estudante)

Usa esta verificação para confirmar se assimilaste todos os conceitos arquiteturais e práticos do processamento de ficheiros em React:

- [ ] **Validação em Dupla Camada**: Compreendo a diferença entre a filtragem de UI via atributo `accept` e a validação em runtime no manipulador `handleFileChange` para eventos de *drag-and-drop*.
- [ ] **Isolamento de Thread com Web Workers**: Entendo como o `pdfjs-dist` utiliza a propriedade `GlobalWorkerOptions.workerSrc` para executar o parsing em segundo plano sem congelar a UI.
- [ ] **Declarações Ambientais TypeScript**: Sei criar ficheiros `.d.ts` (ex.: `mammoth.d.ts`) usando `declare module` para suprir a ausência de tipos em bibliotecas JavaScript.
- [ ] **Parsing Espacial de PDF**: Compreendo como analisar variações da coordenada vertical Y (`item.transform[5]`) para converter glifos visuais em quebras de linha (`\n`) e parágrafos (`\n\n`).
- [ ] **Sanitização Sequencial**: Sei aplicar regras de expressão regular para eliminar caracteres de controlo, remover artefatos binários, unir palavras hifenizadas e normalizar parágrafos.
- [ ] **Sanitização de Nomes de Ficheiro**: Compreendo a importância de substituir separadores (- e _) por espaços (`' '`) ao gerar títulos de documentos a partir do nome do ficheiro.
- [ ] **Cálculo Diferenciado de Métricas**: Distingo a contagem de páginas reais de um PDF do cálculo estimado pela norma ABNT (~280 palavras/página) para ficheiros DOCX e TXT.

Relatório de Engenharia: Migração da Extração de Documentos de Regex Heurístico para Processamento Robusto com PDF.js e Mammoth.js

1. Contexto Técnico e Objetivos de Engenharia

A evolução contínua da ferramenta de análise antiplágio para Trabalhos de Conclusão de Curso (TCC) exigiu uma revisão profunda da arquitetura do módulo de ingestão e extração de texto. Anteriormente, a fiabilidade do sistema estava comprometida por falhas recorrentes na extração de conteúdos de ficheiros carregados pelos utilizadores, gerando ruído nos dados enviados para verificação e impactando negativamente as análises semânticas subsequentes. Para garantir a integridade operacional e elevar a precisão do motor de auditoria, tornou-se imperativo refatorizar a pipeline de extração de documentos, substituindo mecanismos frágeis baseados em parsing manual por bibliotecas consolidadas e testadas no ecossistema de software.

A motivação central por trás do commit `5108169` reside no abandono definitivo de um método arcaico de leitura de ficheiros — baseado em expressões regulares e decodificação manual de streams — em favor da integração das bibliotecas `pdfjs-dist` (PDF.js da Mozilla) e `mammoth`

(Mammoth.js). Esta mudança estratégica visa eliminar falhas de parsing no cliente (*browser*) e garantir que a entrada de dados para o modelo de análise seja constituída por texto limpo, coeso e estruturado.

Os objetivos concretos desta reformulação técnica incluem:

- **Suporte Alargado a Múltiplos Formatos:** Expansão da capacidade de ingestão para aceitar nativamente ficheiros PDF (`.pdf`), documentos do Microsoft Word (`.docx`, `.doc`) e ficheiros de texto simples (`.txt`).
- **Higienização e Sanitização Avançada de Texto:** Implementação de um algoritmo dedicado (`cleanExtractedText`) para eliminar artefactos binários, caracteres de controlo, quebras artificiais de linha e hifenizações de fim de página.
- **Melhoria da Estimativa de Páginas para Auditoria ABNT:** Cálculo do número de páginas reais em PDFs e estimativa automatizada para documentos Word/TXT com base nas métricas formais de diagramação da norma ABNT NBR 14724.

Estabelecidos estes objetivos estratégicos, a secção seguinte apresenta uma análise comparativa aprofundada entre as deficiências estruturais do mecanismo legado e a robustez da nova arquitetura baseada em parsers estruturados.

2. Análise Comparativa de Arquitetura: Módulo Legado vs. Nova Solução

A transição arquitetural realizada representa um salto qualitativo, evoluindo de um modelo passível de falhas baseado na correspondência de padrões (*pattern matching*) por expressões regulares para uma pipeline moderna de parsing estruturado. Enquanto a abordagem antiga tentava interpretar diretamente a estrutura interna de ficheiros binários no browser, a nova solução delega a conversão do documento para motores especializados baseados em Árvores de Sintaxe Abstrata (AST) e no Modelo do Objeto do Documento (DOM).

2.1. Fragilidades do Mecanismo Legado (Regex & Stream Matching)

A implementação legada contida no serviço `src/services/antiplagiarismService.ts` operava através da leitura manual do ficheiro como um `ArrayBuffer`, convertendo-o bloco a bloco numa string binária bruta (*raw string*). Sobre este conteúdo, o sistema aplicava expressões regulares para isolar blocos de texto delimitados pelos operadores PDF `BT` (*Begin Text*) e `ET` (*End Text*), extraindo strings contidas entre parênteses através dos operadores de exibição `Tj` e `TJ`.

Esta metodologia heurística apresentava severas limitações técnicas:

- **Falha em Codificações Não-Padronizadas e CMap:** Documentos PDF que utilizavam mapeamentos de fontes personalizados (*ToUnicode CMap*), fontes embutidas parciais ou codificações de caracteres não-ASCII falhavam na extração, resultando em sequências de caracteres corrompidos ou ilegíveis.
- **Quebra de Estrutura e Parágrafos:** A extração por regex ignorava completamente a posição topológica dos elementos na página, fundindo linhas de tabelas, cabeçalhos, rodapés e parágrafos sem critério de ordenação espacial.
- **Fallback Ineficiente:** Quando a extração via operadores `BT...ET` falhava (gerando menos de 50 caracteres), o sistema recorria a uma expressão regular ASCII de recurso:

`/[A-ZÀ-Úa-zà-ú0-9\s.,;:\-()"'!?]{20,}/g`

Este mecanismo de emergência capturava sequências arbitrárias de texto legível, mas falhava frequentemente ao incluir metadados internos do PDF (como instâncias de `/Type` e `/Font`) ou ao ignorar blocos inteiros de texto válido codificados fora do intervalo ASCII padrão.

2.2. A Nova Arquitetura de Processamento por Biblioteca

A nova arquitetura elimina a necessidade de parsing manual ao incorporar motores de conversão consolidados. A tabela abaixo resume as diferenças fundamentais entre os dois modelos:

Critério de Avaliação	Abordagem Legada (Regex & Stream)	Nova Solução (PDF.js + Mammoth.js)
Formatos Suportados	Apenas PDF (suporte parcial e instável)	PDF (`.pdf`), Word (`.docx`, `.doc`) e Texto (`.txt`)
Fiabilidade da Extração	Baixa; suscetível a erros de codificação e streams comprimidos	Elevada; interpretação nativa da estrutura interna dos ficheiros
Preservação de Estrutura	Inexistente; perda de parágrafos e ordenação de texto	Excelente; reconstrução de linhas e parágrafos por coordenadas espaciais e elementos DOM
Tratamento de Codificação	Frágil; dependente de caracteres legíveis ASCII/Latin1	Robusto; gestão automática de UTF-8, fontes embutidas e CMap
Manutenibilidade do Código	Deficiente; código complexo de expressões regulares difíceis de testar	Alta; código modular, tipado em TypeScript com abstração de bibliotecas

A delegação do parsing a bibliotecas consolidadas reduz drasticamente o risco de exceções em tempo de execução (*runtime errors*). O processamento de PDFs passa a ser gerido pela engine da Mozilla, enquanto os ficheiros Word são descompilados pelo `mammoth.js`, garantindo a integridade dos dados extraídos antes do seu envio para as etapas de análise.

Analisadas as diferenças arquiteturais, a secção seguinte detalha os aspetos de implementação técnica e a mecânica interna da nova solução.

3. Implementação Técnica e Mecanismos da Nova Solução

A integração da nova pipeline no ecossistema TypeScript/React exigiu uma estruturação cuidadosa de dependências, gestão de concorrência no browser e algoritmos de sanitização de dados.

3.1. Gestão de Workers e Concorrência no PDF.js

O parsing de PDFs no browser envolve a interpretação de tabelas de objetos e a descompressão de streams em tempo real. Para evitar o bloqueio da thread principal de interface (UI thread), a biblioteca `pdfjs-dist` delega estas operações computacionais pesadas para um *Web Worker*. No serviço `src/services/antiplagiarismService.ts`, a inicialização do worker foi configurada de forma resiliente:

```
import * as pdfjsLib from 'pdfjs-dist';
import pdfjsWorker from 'pdfjs-dist/build/pdf.worker.min.mjs?url';

if (typeof window !== 'undefined' && pdfjsLib.GlobalWorkerOptions) {
  try {
    pdfjsLib.GlobalWorkerOptions.workerSrc = pdfjsWorker;
  } catch {
    pdfjsLib.GlobalWorkerOptions.workerSrc    =    `https://unpkg.com/pdfjs-
dist@${pdfjsLib.version}/build/pdf.worker.min.mjs`;
  }
}
```

Caso a importação local do worker falhe devido a restrições do bundler no ambiente de produção, o sistema ativa um *fallback* transparente via CDN (`unpkg`).

Durante a extração, o método `extractTextFromPdfFile` itera por cada página do documento (`pdfDoc.getPage`). A reconstituição topológica do texto é realizada através da análise do vetor de transformação afim bidimensional (2D\text{ affine transformation matrix}) fornecido pela propriedade `item.transform`. Na geometria das páginas PDF, a matriz de transformação afim de 3 \times 3 coordenadas é representada por [a, b, c, d, e, f], onde o elemento no índice 5 (`transform[5]`) corresponde à translação vertical no eixo Y (matriz linha 3, coluna 2, assumindo a origem no canto inferior esquerdo da página):

- **Quebra de Parágrafo:** Se a variação na coordenada vertical Y entre dois itens consecutivos for superior a 14 unidades (`Math.abs(currentY - lastY) > 14`), insere-se uma quebra dupla de linha (`\n\n`).
- **Quebra de Linha Simples:** Se o delta vertical for superior a 5 unidades (`Math.abs(currentY - lastY) > 5`), insere-se uma quebra simples (`\n`).
- **Espaçamento Horizontal:** Se os elementos pertencerem à mesma linha, junta-se o texto adicionando espaços delimitadores adequados.

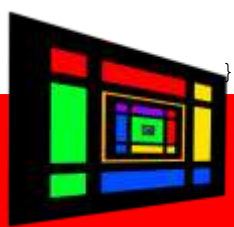
3.2. Suporte a Word (.docx, .doc), Texto (.txt) e Tipagem em TypeScript

O suporte a documentos Word é provido pelo `mammoth.js`, que converte os ficheiros `.docx` extraindo o texto bruto diretamente da sua estrutura XML comprimida (`word/document.xml`). Como a biblioteca `mammoth` não disponibilizava definições de tipos nativas compatíveis com as configurações estritas do compilador TypeScript (`tsc`), foi criado o ficheiro de declarações de ambiente `src/types/mammoth.d.ts` para evitar erros em tempo de compilação:

```
declare module 'mammoth' {
  export interface MammothOptions {
    arrayBuffer?: ArrayBuffer;
    buffer?: any;
    path?: string;
  }

  export interface MammothMessage {
    type: string;
    message: string;
  }

  export interface MammothResult {
    value: string;
    messages: MammothMessage[];
  }
```



```
  export function extractRawText(options: MammothOptions):
Promise<MammothResult>;
  export function convertToHtml(options: MammothOptions):
Promise<MammothResult>;
  export function convertToMarkdown(options: MammothOptions):
Promise<MammothResult>;

  const mammoth: {
    extractRawText: typeof extractRawText;
    convertToHtml: typeof convertToHtml;
    convertToMarkdown: typeof convertToMarkdown;
  };

  export default mammoth;
}
```

A orquestração do roteamento de ficheiros é realizada pela função `extractTextFromDocument`, que valida o tipo de documento através da extensão do nome, do tipo MIME e de uma inspeção da assinatura binária (*magic bytes*):

```
static async extractTextFromDocument(file: File): Promise<{
  text: string;
  pagesEstimate: number;
  format: 'pdf' | 'docx' | 'txt';
}> {
  const name = file.name.toLowerCase();
  const mime = file.type.toLowerCase();

  if (name.endsWith('.pdf') || mime.includes('pdf')) {
    const res = await this.extractTextFromPdfFile(file);
    return { ...res, format: 'pdf' };
  }

  if (name.endsWith('.docx') || name.endsWith('.doc') ||
mime.includes('word') || mime.includes('officedocument')) {
    const res = await this.extractTextFromDocxFile(file);
    return { ...res, format: 'docx' };
  }

  if (name.endsWith('.txt') || mime.includes('text/plain')) {
    const raw = await file.text();
    const clean = this.cleanExtractedText(raw);
    const wordsCount = clean.split(/\s+/).filter(Boolean).length;
    return {
      text: clean,
      pagesEstimate: Math.max(1, Math.ceil(wordsCount / 280)),
      format: 'txt',
    };
  }

  // Tenta PDF se a assinatura binária contiver %PDF
  try {
    const slice = await file.slice(0, 5).text();
    if (slice.startsWith('%PDF')) {
      const res = await this.extractTextFromPdfFile(file);
      return { ...res, format: 'pdf' };
    }
  } catch {
    // Fallback gracioso
```

```
  }

  // Fallback para Mammoth.js
  try {
    const res = await this.extractTextFromDocxFile(file);
    return { ...res, format: 'docx' };
  } catch {
    throw new Error('Formato de arquivo não suportado. Por favor envie um
documento no formato PDF (.pdf) ou Word (.docx).');
  }
}
```

3.3. Pipeline de Sanitização de Texto (`cleanExtractedText`)

O texto bruto extraído de qualquer uma das fontes passa obrigatoriamente pelo método estático `cleanExtractedText`, responsável por eliminar ruídos de extração através de uma pipeline sequencial composta por 5 etapas minuciosamente definidas:

1. **Remoção de Caracteres de Controlo:** Elimina caracteres nulos e de controlo não-imprimíveis através da expressão regular `/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]/g`, preservando as tabulações (`\t`) e as quebras de linha (`\n`).
2. **Normalização de Quebras de Linha:** Padroniza quebras de linha do Windows (`\r\n`) e Mac antigo (`\r`) para o padrão Unix (`\n`).
3. **Eliminação de Artefactos Binários:** Remove resquícios de cabeçalhos e objetos PDF não processados com a regex `/^(?:%PDF-[\d.]+|<<\/[\w\s/]+>>|obj\b[\s\S]*?endobj|stream[\s\S]*?endstream)/gm`.
4. **Desfazer Hifenização de Fim de Linha:** Junta palavras divididas na transição de linha (ex.: transformando `"sustenta-\nbilidade"` em `"sustentabilidade"`) via `/([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])/g` com a substituição por `$1$2`.
5. **Normalização Estrutural de Parágrafos:** Divide o texto em blocos separados por múltiplas linhas em branco (`.split(/\n\s*\n+/)`). Para cada bloco, efetua um split interno por quebras de linha simples (`paragraph.split('\n')`), aplica `.map(l => l.trim())` em cada fragmento, remove segmentos vazios com `.filter(Boolean)`, une os fragmentos com um espaço em branco, e substitui espaços consecutivos ou caracteres de espaço não-quebrável (*non-breaking spaces*) através de `.replace(/[ \t\u00A0]+/g, ' ')`. Por fim, descarta parágrafos vazios com `.filter(p => p.length > 0)` e recombina os blocos válidos utilizando quebras duplas de linha (`.join('\n\n')`).

Detalhados os aspetos de implementação técnica, a secção seguinte aborda o impacto da solução no tratamento de exceções, na qualidade para auditoria e na interface do utilizador.

4. Robustez, Gestão de Erros e Impacto na Qualidade para Auditoria

A qualidade do texto extraído afeta diretamente a precisão dos algoritmos de deteção de similaridade e as chamadas à API do modelo Gemini. A ausência de ruído binário e a preservação dos parágrafos garantem que as janelas de contexto enviadas para análise sintática e semântica reflitam fidedignamente o conteúdo do TCC.

4.1. Tratamento defensivo e PDFs sem camada de texto

O sistema implementa tratamento defensivo contra ficheiros corrompidos ou inviáveis. Uma limitação comum no processamento de PDFs ocorre quando o ficheiro enviado consiste numa digitalização por imagem (ex.: digitalização por scanner sem camada OCR).

Nesses cenários, o parser `pdfjs-dist` executa a leitura sem gerar exceções de runtime, contudo a lista de itens de texto resulta vazia. O sistema deteta esta condição e fornece uma resposta clara na interface:

"O arquivo PDF foi carregado com sucesso pelo PDF.js, mas não possui camadas de texto selecionáveis legíveis (documento baseado apenas em imagens escaneadas). Cole o texto do TCC diretamente no campo de texto para prosseguir."

4.2. Métricas de Paginação e Estimativa ABNT

Para efeitos de verificação da conformidade académica, o sistema calcula a dimensão real do documento com base nas diretrizes formais de apresentação de trabalhos académicos:

- **Documentos PDF:** A contagem de páginas é extraída diretamente do metadado do ficheiro via `pdfDoc.numPages`.
- **Documentos DOCX e TXT:** Como os ficheiros Word e TXT não contêm uma paginação fixa intrínseca antes da renderização gráfica, o sistema aplica um cálculo de estimativa baseado na norma **ABNT NBR 14724**. Considerando as especificações padrão da norma (fonte tamanho 12, espaçamento interlinear de 1,5, margens superior/esquerda de 3 cm e inferior/direita de 2 cm), obtém-se uma média aproximada de **280 palavras por página padrão**. A estimativa é calculada pela fórmula:

\text{Páginas Estimadas} = \max\left(1, \left\lceil \frac{\text{Contagem Total de Palavras}}{280} \right\rceil\right)

4.3. Atualizações na Interface do Utilizador (UI)

A componente React `src/components/AntiplagiarismTccView.tsx` foi atualizada para expor o estado do processamento do ficheiro ao utilizador:

- **Badges Informativos por Formato:** Exibição de identificadores visuais que indicam a engine utilizada no parsing:
  - `PDF (PDF.js Mozilla)`
  - `Word DOCX (Mammoth.js)`
  - `Documento de Texto`
- **Métricas de Volume:** Exibição do tamanho do ficheiro carregado em KB, juntamente com o número de páginas (reais para PDF e estimadas ABNT para DOCX/TXT) e a contagem total de palavras processadas.
- **Indicadores visuais de estado:** Integração do componente de carregamento animado `<Loader2 size={32} className="animate-spin text-blue-600" />` durante o parsing do ficheiro.
- **Confirmação de Higienização:** Apresentação do ícone `CheckCircle2` junto à área de pré-visualização do texto, indicando o número exato de palavras e parágrafos higienizados e prontos para a auditoria antiplágio.

Com a consolidação da interface e do tratamento de erros, a secção final sintetiza os ganhos técnicos e apresenta recomendações de engenharia para evoluções futuras.

5. Conclusão e Recomendações de Engenharia

A refatorização do módulo de extração de documentos efetuada no commit `5108169` eliminou as fragilidades associadas ao parsing por expressões regulares heurísticas. A adoção de bibliotecas especializadas e a implementação da pipeline de sanitização asseguram a alta disponibilidade e a precisão do sistema de verificação antiplágio.

Ganhos Técnicos Consolidados

1. **Fiabilidade na Extração:** Eliminação de falhas na leitura de PDFs comprimidos ou com codificações de fonte complexas através do motor `pdfjs-dist`.
2. **Suporte Multiformato Naitvo:** Capacidade de ingestão direta de ficheiros do Microsoft Word (`.docx`, `.doc`) via `mammoth.js` e ficheiros de texto simples (`.txt`).
3. **Sanitização de Dados:** Garantia de texto limpo, sem artefactos binários ou hifenizações incorretas através da função `cleanExtractedText`.
4. **Manutenibilidade do Código:** Introdução do ficheiro de declarações `src/types/mammoth.d.ts`, permitindo uma integração estritamente tipada com o compilador TypeScript.
5. **Transparência para o Utilizador:** Interface com feedback em tempo real, indicando o formato do ficheiro, o tamanho em KB, a contagem de palavras e a estimativa de páginas ABNT.

Recomendações de Engenharia Futuras

- **Integração de Camada de OCR no Cliente:** Como evolução técnica para mitigar a limitação de PDFs baseados exclusivamente em imagens digitalizadas, recomenda-se a integração de uma biblioteca de OCR executada no cliente (como o `Tesseract.js`) enquanto mecanismo de *fallback* secundário, ativado automaticamente quando `pdfDoc` não retornar caracteres selecionáveis.

Documento de Arquitetura de Software: Subsistema de Ingestão e Processamento de Documentos Anti-Plágio

1. Visão Geral e Objetivos do Subsistema de Ingestão

1.1 Contexto da Arquitetura e Objetivos de Negócio

A reestruturação do subsistema de ingestão de documentos no pipeline anti-plágio representa uma evolução estratégica fundamental para assegurar a fiabilidade da auditoria académica e a precisão da análise executada pelos modelos de linguagem (especificamente a API do Google Gemini). Anteriormente, a extração de texto assentava num mecanismo simplificado e frágil baseado em expressões regulares aplicadas diretamente sobre as sequências de bytes de ficheiros PDF. Essa abordagem causava falhas recorrentes de parsing em documentos complexos, perda da estrutura de parágrafos, corrupção de caracteres codificados e incapacidade de processar o formato Microsoft Word (`.docx`). Ao transitar de um parsing simplificado para uma arquitetura robusta suportada por bibliotecas especializadas do ecossistema JavaScript/TypeScript (`pdfjs-dist` e `mammoth`), o sistema passa a fornecer dados

higienizados e bem estruturados, impactando diretamente a qualidade do cálculo de similaridade e a assertividade do motor de IA.

A fragilidade da arquitetura legada decorria da tentativa de capturar blocos de texto PDF via regex através da correspondência manual dos operadores `BT...ET`, `Tj` e `TJ`. Essa técnica ignorava elementos críticos da especificação PDF, tais como mapas de codificação de fontes (*ToUnicode CMaps*), subconjuntos de fontes (*font subsetting*), objetos de *stream* comprimidos por algoritmo Deflate e posicionamentos de glifos não-lineares. Quando o regex falhava, o sistema recorria a varreduras heurísticas de sequências ASCII/Latin1, resultando na perda da sequência lógica de frases, na fragmentação de parágrafos e em leituras incompletas. Para resolver esses riscos operacionais, o novo subsistema estabelece os seguintes objetivos primários:

1. **Suporte Nativo Multi-Formato:** Suporte arquitetural completo para documentos PDF (via `pdfjs-dist` da Mozilla) e Microsoft Word `.docx` (via `mammoth`), além de arquivos de texto simples (`.txt`).
2. **Higienização Profunda de Dados:** Implementação do algoritmo universal `cleanExtractedText`, projetado para eliminar caracteres de controlo, artefatos de *stream*, desfazer hifenizações de fim de linha e reconstruir a coesão de parágrafos para consumo otimizado por LLMs.
3. **Métricas de Conformidade Académica:** Cálculo automatizado de estimativa de páginas em conformidade com as normas ABNT NBR 14724 para documentos Word e TXT, e leitura do número de páginas reais para arquivos PDF.

Estas decisões de arquitetura conectam os objetivos de negócio — como a ampliação dos formatos suportados e a garantia do rigor na auditoria académica — a uma implementação técnica de alto desempenho no lado do cliente.

1.2 Âmbito das Alterações de Código e Arquitetura Multi-Formato

O alcance técnico das modificações consolidadas no commit `5108169` abrange a substituição completa da camada de extração legada, a introdução de declarações de tipos estáticos e a atualização dos componentes da camada de apresentação (UI). A intervenção totalizou **326 adições e 102 remoções** distribuídas por 5 ficheiros do repositório, isolando as responsabilidades de extração e higienização na camada de serviço para garantir que a interface reaja a dados unificados e agnósticos do formato de origem.

A tabela seguinte detalha os ficheiros modificados, as suas responsabilidades no subsistema e as métricas de linhas de código alteradas:

Ficheiro	Responsabilidade no Subsistema	Métricas de Linhas
`package.json`	Gestão de dependências externas de produção (`mammoth` e `pdfjs-dist`).	+2 / -0
`bun.lock`	Bloqueio de versões exatas e árvore de dependências no ambiente de execução Bun.	+48 / -0

`src/types/mammoth.d.ts`	Declaração de tipos TypeScript para suporte e tipagem estática do módulo `mammoth`.	+29 / -0
`src/services/antiplagiarismService.ts`	Camada central de serviço, algoritmos de extração, higienização e roteamento polimórfico.	+180 / -75
`src/components/AntiplagiarismTccView.tsx`	Camada de apresentação, gestão de estados de upload, badges de formato e pré-visualização.	+67 / -27

A expansão do alcance do pipeline para processar ficheiros `.docx` juntamente com `.pdf` e `.txt` elimina a barreira operacional que exigia que o utilizador convertesse previamente o seu trabalho académico para PDF. O subsistema assume a responsabilidade de tratar as especificidades estruturais de cada formato internamente, entregando um fluxo de texto limpo e padronizado às fases subsequentes de verificação de plágio.

Esta reformulação dos componentes exige uma análise detalhada da infraestrutura de dependências e das configurações de tipagem adotadas no projeto.

2. Infraestrutura, Dependências e Gestão de Tipos

2.1 Mapeamento de Dependências Externas (`package.json`)

A seleção das dependências para o processamento de documentos no lado do cliente (*browser-side parsing*) foi pautada pela necessidade de utilizar bibliotecas maduras e de alto desempenho no ecossistema JavaScript/TypeScript. A transferência do processamento do servidor para o navegador reduz a latência do pipeline, elimina custos de infraestrutura de computação de backend para conversão de ficheiros e garante a privacidade dos documentos académicos antes do envio final para auditoria.

O ficheiro `package.json` foi atualizado com as seguintes dependências de produção:

- **`pdfjs-dist` (versão `^6.4.299`):** Biblioteca oficial mantida pela Mozilla para parsing estruturado de documentos PDF. O seu motor de renderização e extração acede diretamente às tabelas de fontes, aos mapas de caracteres e à árvore de objetos do documento.
- **`mammoth` (versão `^1.13.0`):** Conversor especializado na extração de texto puro a partir de documentos Word `.docx` (OpenXML). A sua escolha fundamenta-se na capacidade de ignorar estilos visuais complexos, tabelas aninhadas e elementos decorativos, focando-se estritamente na extração do texto do documento.

Ao efetuar o *parsing* diretamente no *browser*, o subsistema evita o upload desnecessário de arquivos binários pesados para servidores intermediários, processando apenas a camada de texto estritamente necessária.

2.2 Configuração do Worker do PDF.js e Tipagem do Mammoth (`mammoth.d.ts`)

O processamento de arquivos PDF extensos exige a execução de rotas assíncronas isoladas da thread principal da interface do utilizador, evitando o bloqueio da renderização React via Web

Workers. Adicionalmente, a utilização de bibliotecas sem tipagem nativa no TypeScript requer o mapeamento de interfaces para assegurar a integridade da compilação.

No ficheiro `src/services/antiplagiarismService.ts`, a inicialização do worker do PDF.js foi projetada com um mecanismo de resiliência (*fallback*) dinâmico:

```
import * as pdfjsLib from 'pdfjs-dist';
import pdfjsWorker from 'pdfjs-dist/build/pdf.worker.min.mjs?url';

// Configura o worker do PDF.js do Mozilla de forma segura
if (typeof window !== 'undefined' && pdfjsLib.GlobalWorkerOptions) {
  try {
    pdfjsLib.GlobalWorkerOptions.workerSrc = pdfjsWorker;
  } catch {
    pdfjsLib.GlobalWorkerOptions.workerSrc    =    `https://unpkg.com/pdfjs-
dist@${pdfjsLib.version}/build/pdf.worker.min.mjs`;
  }
}
```

A estratégia prioriza o carregamento do worker local via importação de URL estática (`?url`). Caso ocorra uma falha na resolução pelo empacotador em tempo de execução no ambiente do navegador (`typeof window !== 'undefined'`), o bloco `catch` injeta condicionalmente o caminho público do CDN `unpkg` correspondente à versão exata da biblioteca.

Dado que o pacote `mammoth` não possui tipos TypeScript nativos, foi criado o ficheiro de declaração `src/types/mammoth.d.ts` para prover segurança de tipos estática:

```
declare module 'mammoth' {
  export interface MammothOptions {
    arrayBuffer?: ArrayBuffer;
    buffer?: any;
    path?: string;
  }

  export interface MammothMessage {
    type: string;
    message: string;
  }

  export interface MammothResult {
    value: string;
    messages: MammothMessage[];
  }

  export       function       extractRawText(options:       MammothOptions):
Promise<MammothResult>;
  export       function       convertToHtml(options:       MammothOptions):
Promise<MammothResult>;
  export      function      convertToMarkdown(options:      MammothOptions):
Promise<MammothResult>;

  const mammoth: {
    extractRawText: typeof extractRawText;
    convertToHtml: typeof convertToHtml;
    convertToMarkdown: typeof convertToMarkdown;
  };

  export default mammoth;
```

```
}
```

Com o suporte de infraestrutura e tipos devidamente estabelecido, a análise avança para a lógica do algoritmo universal de limpeza de texto.

3. Arquitetura do Serviço Principal (`AntiplagiarismService`)

3.1 Algoritmo Universal de Higienização de Texto (`cleanExtractedText`)

A precisão dos modelos de linguagem e dos algoritmos de verificação ABNT depende da remoção de ruídos de extração e da reconstituição da estrutura lógica dos parágrafos. O método estático `cleanExtractedText` atua como a camada central de higienização de dados do subsistema.

```
static cleanExtractedText(rawText: string): string
```

A sequência do algoritmo executa 5 operações encadeadas:

1. **Filtragem de Caracteres de Controlo:** Remoção de bytes nulos e caracteres não-imprimíveis através da regex `[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]`, preservando intencionalmente os tabuladores (`\t`) e quebras de linha (`\n`).
2. **Padronização de CR/LF:** Substituição unificada de quebras de linha no formato Windows (`\r\n`) e Mac antigo (`\r`) por quebras Unix (`\n`).
3. **Remoção de Artefatos de PDF/Streams:** Eliminação de resquícios de objetos binários e marcas de cabeçalho através da expressão regular multilinha `/^(?:%PDF-[\d.]+|<<\/[\w\s/]+>>|obj\b[\s\S]*?endobj|stream[\s\S]*?endstream)/gm`.
4. **Desfazer Hifenização de Quebra de Linha:** Recomposição de palavras interrompidas no final da linha (ex: `"sustenta-\nção"` é unificada para `"sustentação"`). A expressão regular `([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])` substitui a ocorrência pelos grupos capturados `$1$2`.
5. **Reconstrução e Normalização de Parágrafos:** Divisão do texto bruto em blocos através de quebras duplas (`\n\s*\n+`). Cada parágrafo é processado individualmente: as suas linhas internas são unidas com espaço (`.join(' ')`), os espaços internos duplicados ou inseparáveis Unicode são colapsados (`.replace(/[ \t\u00A0]+/g, ' ')`), e os blocos limpos são reunidos por quebras duplas (`\n\n`).

Abaixo apresenta-se a implementação exata contida no serviço:

```
static cleanExtractedText(rawText: string): string {
  if (!rawText) return '';

  return rawText
    // 1. Remove caracteres nulos e de controle não-imprimíveis (mantém \t e
\n)
    .replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]/g, '')
    // 2. Normaliza quebras de linha Windows/Mac
    .replace(/\r\n/g, '\n')
    .replace(/\r/g, '\n')
    // 3. Remove resquícios de streams ou artefatos binários
    .replace(/^(?:%PDF-
[\d.]+|<<\/[\w\s/]+>>|obj\b[\s\S]*?endobj|stream[\s\S]*?endstream)/gm, '')
    // 4. Une palavras quebradas por hífen no final de linhas (ex: "sustenta-
\nbilidade" -> "sustentabilidade")
    .replace(/([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])/g, '$1$2')
    // 5. Normaliza quebras simples no meio de frases em parágrafos contínuos
```

```
    .split(/\n\s*\n+/)
    .map((paragraph) => {
      return paragraph
        .split('\n')
        .map((l) => l.trim())
        .filter(Boolean)
        .join(' ')
        .replace(/[ \t\u00A0]+/g, ' ')
        .trim();
    })
    .filter((p) => p.length > 0)
    .join('\n\n');
}
```

Esta padronização assegura que a extração a partir de PDFs possa prosseguir com previsibilidade espacial e estrutural.

3.2 Ingestão e Extração de Documentos PDF (`extractTextFromPdfFile`)

A extração de documentos PDF via `pdfjs-dist` é realizada de forma assíncrona. O ficheiro é convertido num `ArrayBuffer` e transmitido ao motor do PDF.js via `pdfjsLib.getDocument({ data: new Uint8Array(arrayBuffer), useSystemFonts: true })`. O serviço itera iterativamente sobre todas as páginas do documento (`numPages`).

Para reconstruir a disposição espacial do texto, o algoritmo inspeciona a matriz de transformação afim 2D de cada item de texto (`item.transform`). Na especificação PDF, `transform` é um vetor de 6 elementos [a, b, c, d, e, f], no qual o índice `5` (f) representa o deslocamento de tradução na coordenada vertical Y no espaço de coordenadas da página:

```
const currentY = item.transform ? item.transform[5] : undefined;
if (lastY !== undefined && currentY !== undefined && Math.abs(currentY -
lastY) > 5) {
  if (Math.abs(currentY - lastY) > 14) {
    pageStr += '\n\n';
  } else {
    pageStr += '\n';
  }
}  else  if  (pageStr.length  >  0  &&  !pageStr.endsWith('  ')  &&
!pageStr.endsWith('\n')) {
  pageStr += ' ';
}
```

- **Diferença |\Delta Y| > 5:** Identifica a transição de linha dentro do mesmo bloco de texto, inserindo uma quebra simples (`\n`).
- **Diferença |\Delta Y| > 14:** Identifica uma alteração substancial de bloco vertical, interpretada como mudança de parágrafo, inserindo uma quebra dupla (`\n\n`).
- **Inexistência de Variação Vertical:** Se o item estiver na mesma linha horizontal, garante-se que os termos separados por espaço sejam unidos corretamente via `pageStr += ' '`.

Caso o PDF seja constituído exclusivamente por imagens escaneadas (sem camada de texto selecionável), a string limpa resultante do processo terá comprimento zero. O sistema trata essa exceção devolvendo uma mensagem de orientação:



"O arquivo PDF foi carregado com sucesso pelo PDF.js, mas não possui camadas de texto selecionáveis legíveis (documento baseado apenas em imagens escaneadas). Cole o texto do TCC diretamente no campo de texto para prosseguir."

3.3 Ingestão e Extração de Documentos Word (`extractTextFromDocxFile`)

A extração de ficheiros Microsoft Word `.docx` é realizada pelo método `extractTextFromDocxFile`, que utiliza a biblioteca `mammoth.js`. A extração foca-se na conversão da estrutura OpenXML para texto bruto legível:

```
static async extractTextFromDocxFile(file: File): Promise<{ text: string; pagesEstimate: number }> {
  try {
    const arrayBuffer = await file.arrayBuffer();
    const result = await mammoth.extractRawText({ arrayBuffer });
    const cleanText = this.cleanExtractedText(result.value || '');

    // Estimativa de páginas ABNT: média de ~280 palavras por página padrão (NBR 14724)
    const wordsCount = cleanText.split(/\s+/).filter(Boolean).length;
    const pagesEstimate = Math.max(1, Math.ceil(wordsCount / 280));

    if (cleanText.length > 0) {
      return {
        text: cleanText,
        pagesEstimate,
      };
    }

    return {
      text: 'O arquivo .docx foi processado com mammoth.js, mas não continha texto legível.',
      pagesEstimate: 1,
    };
  } catch (err: any) {
    console.error('[AntiplagiarismService] Erro ao extrair DOCX com mammoth.js:', err);
    throw new Error(`Falha ao converter o documento Word .docx com mammoth.js: ${err?.message || err}`);
  }
}
```

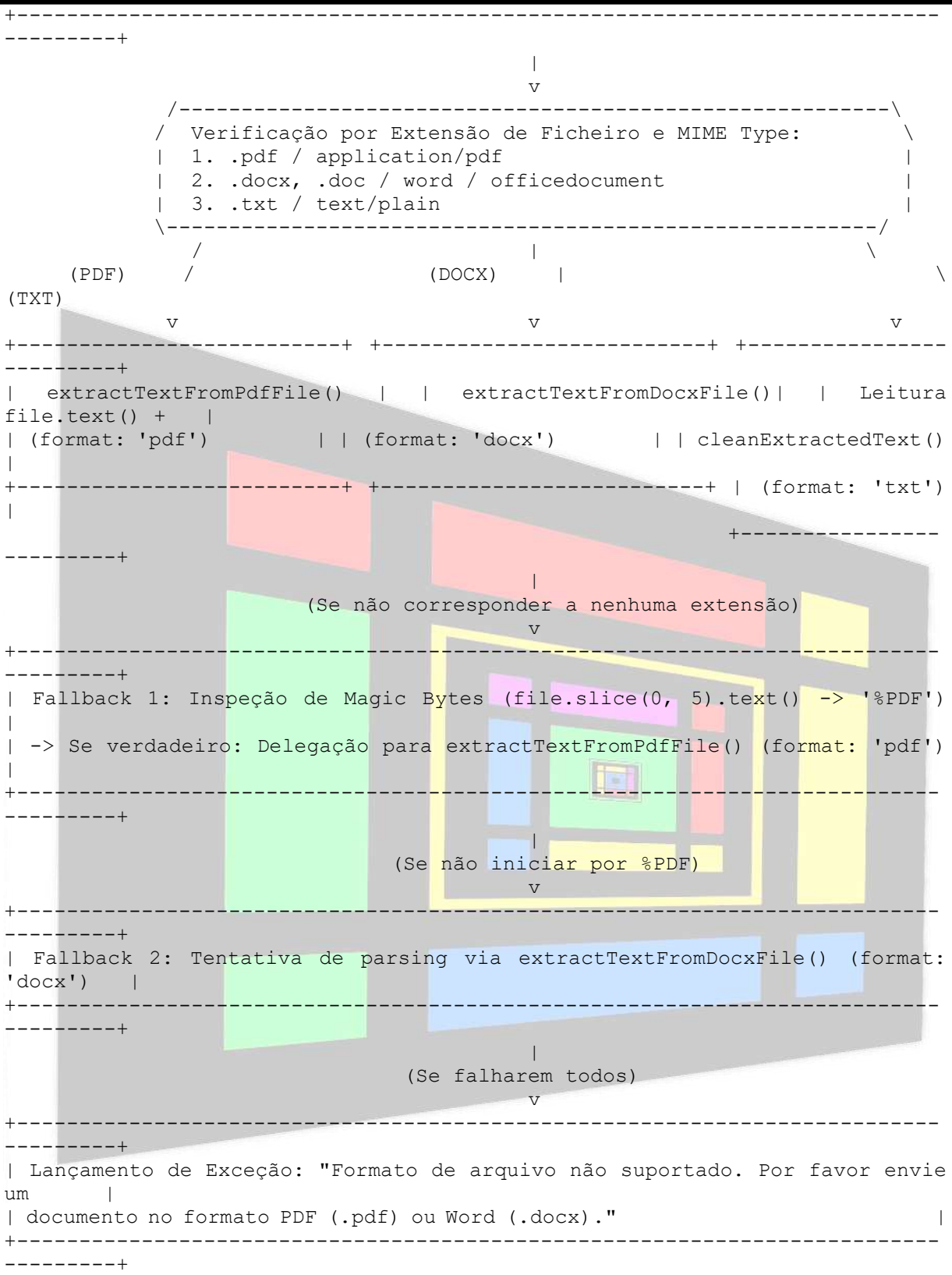
Quando o documento `.docx` é processado mas não possui conteúdo extraível, o serviço devolve a mensagem informando que o ficheiro foi processado via `mammoth.js`, mas não continha texto legível, atribuindo uma estimativa base de 1 página.

3.4 Ponto de Entrada Unificado e Fallback Polimórfico (`extractTextFromDocument`)

Para simplificar a integração com a camada de apresentação, o `AntiplagiarismService` expõe o método fachada (*Facade*) `extractTextFromDocument`. Este método unifica o roteamento de formatos e estabelece uma estratégia de resiliência por inspeção de assinatura de ficheiros (*Magic Bytes*).

```
+-------------------------------------------------------------------------
---------+
|                    Ficheiro Submetido (File Object)                     |
```

UPDATE 5108169



Esta arquitetura garante o processamento de ficheiros renomeados com extensões incorretas através da validação da assinatura `%PDF` e da execução do *fallback* via Mammoth.

4. Estratégia de Estimativa de Páginas ABNT e Métricas

4.1 Lógica de Cálculo ABNT NBR 14724

Nos ficheiros PDF, a contagem de páginas reflete as páginas físicas do documento fornecidas diretamente pelo parâmetro `numPages` da biblioteca `pdfjs-dist`. Para os ficheiros de fluxo contínuo sem paginação explícita — como `.docx` e `.txt` —, o subsistema aplica uma métrica de estimativa baseada nas diretrizes de formatação da ABNT (NBR 14724).

A estimativa do volume de páginas ABNT é calculada pela seguinte fórmula matemática:

\text{Páginas ABNT} = \max\left(1, \left\lceil \frac{\text{Contagem de Palavras}}{280} \right\rceil\right)

A escolha da constante de **280 palavras por página** fundamenta-se na densidade média de texto num trabalho académico formatado segundo as normas ABNT:

- Fonte tamanho 12 (Arial ou Times New Roman).
- Espaçamento entrelinhas de 1,5.
- Margem superior e esquerda de 3 cm; margem inferior e direita de 2 cm.
- Indentação de parágrafos padrão (1,25 cm).

Desta forma, um documento `.docx` extraído com 1.400 palavras resulta em 1400 / 280 = 5 páginas estimadas, garantindo coerência visual e volumétrica na interface do utilizador.

5. Integração na Camada de Apresentação (`AntiplagiarismTccView`)

5.1 Estado do Componente e Manipulação de Upload

O componente de interface `AntiplagiarismTccView.tsx` foi reestruturado para substituir o estado focado exclusivamente em arquivos PDF por um modelo de upload agnóstico ao formato.

As seguintes propriedades e variáveis de estado foram introduzidas:

- **`isExtractingDocument` (boolean):** Substitui o antigo `isExtractingPdf`, controlando os indicadores visuais de carregamento (*spinners*) durante o processamento de qualquer formato.
- **`fileFormat` (`'pdf' | 'docx' | 'txt' | null`):** Armazena o tipo de documento identificado para renderização condicional dos metadados.
- **Filtro de Upload (`<input type="file">`):** A propriedade `accept` do elemento HTML foi atualizada para aceitar extensões e tipos MIME explícitos:

```
accept=".pdf,application/pdf,.docx,application/vnd.openxmlformats-officedocument.wordprocessingml.document,.doc,application/msword,.txt,text/plain"
```

Durante o processamento do ficheiro, o manipulador `handleFileChange` valida o nome do documento. Caso o título do TCC (`tccTitle`) esteja vazio ou corresponda a um dos dois valores padrão do sistema (`'Trabalho de Conclusão de Curso (TCC)'` ou `'A Aplicação de Inteligência Artificial na Gestão de Conhecimento'`), o nome do ficheiro é higienizado pela função `cleanName`:

```
const cleanName = file.name
  .replace(/\.(pdf|docx|doc|txt)$/i, '')
  .replace(/[-_]+/g, '');
setTccTitle(cleanName);
```

Nota de Implementação: A expressão regular `.replace(/[-_]+/g, '')` remove ativamente hífens e sublinhados do nome do ficheiro, convertendo-os diretamente em cadeias contínuas limpas para o título do trabalho.

5.2 Feedback Visual e Exibição de Metadados

A interface fornece ao utilizador indicadores visuais detalhados após a conclusão da extração:

1. **Badge de Formato:** Renderização dinâmica baseada no estado `fileFormat`:
   - `'pdf'`: `PDF (PDF.js Mozilla)`
   - `'docx'`: `Word DOCX (Mammoth.js)`
   - `'txt'`: `Documento de Texto`
2. **Badge de Páginas:**
   - Para PDFs: `${pdfPagesEstimate} páginas reais`
   - Para Word/TXT: `~${pdfPagesEstimate} páginas ABNT`
3. **Badge de Volume de Palavras:** Exibição do contador `${wordCount} palavras extraídas`.

Na caixa de pré-visualização de texto (`<textarea>`), um cabeçalho exibe o ícone `CheckCircle2` em conjunto com a confirmação da ferramenta utilizada (*"Texto limpo extraído com Mammoth.js"*, *"PDF.js (Mozilla)"* ou *"Extrator"*), detalhando o número total de palavras e parágrafos processados. O utilizador mantém acesso de edição direta ao texto na `<textarea>` antes de iniciar a análise de plágio.

6. Diretrizes Técnicas e Próximos Passos para a Engenharia

6.1 Matriz de Riscos, Limitações e Tratamento de Exceções

A matriz abaixo sistematiza os cenários de falha identificados no processamento de documentos no lado do cliente e as respetivas estratégias de mitigação arquitetural:

Ponto de Falha / Risco	Cenário / Impacto	Estratégia de Mitigação Implementada
PDF sem Camada de Texto (Digitalizado)	O PDF.js lê o arquivo mas não encontra elementos de string legíveis nas páginas.	O sistema deteta `cleanText.length === 0` e devolve uma mensagem informativa orientando o utilizador a colar o texto manualmente.
Falha no Web Worker do PDF.js	A resolução da URL local do worker falha devido a configurações do bundler em produção.	Bloco `try/catch` na inicialização de `GlobalWorkerOptions.workerSrc` com *fallback* automático para o CDN `unpkg`.
Arquivos `.doc` antigos (Binary OLE)	O `Mammoth.js` destina-se a arquivos XML `.docx`, falhando em estruturas binárias antigas.	O método captura o erro no processamento de `.doc` e apresenta uma mensagem clara solicitando a conversão para `.docx` ou `.pdf`.

Documentos com Estruturas Complexas	Tabelas, caixas de texto e elementos gráficos podem desorganizar o fluxo de leitura.	Utilização de `extractRawText` no Mammoth e ordenação por coordenadas Y no PDF.js, seguida da higienização por `cleanExtractedText`.

6.2 Recomendações de Qualidade de Código e Manutenção

Para garantir a sustentabilidade e a evolução do subsistema de ingestão, recomenda-se à equipa de engenharia a adoção das seguintes diretrizes:

- **Testes Unitários de Higienização:** Implementar testes automatizados para o método `cleanExtractedText` cobrindo cenários limite (*edge cases*), tais como caracteres nulos (`\x00`), quebras de linha Windows (`\r\n`), cabeçalhos `%PDF-1.7` e palavras hifenizadas no final da linha.
- **Testes de Ingestão de Ficheiros:** Desenvolver uma suíte de ficheiros de teste contendo instâncias `.pdf`, `.docx` e `.txt` com diferentes volumetrias para validar a assertividade do cálculo de palavras e da estimativa ABNT.
- **Monitorização do Tamanho do Bundle:** Avaliar o impacto das bibliotecas `pdfjs-dist` e `mammoth` no tamanho final do pacote estático da aplicação web, utilizando técnicas de *dynamic import* ou *code splitting* para otimizar o tempo de carregamento inicial da interface.

